

Department of Mathematics and Computer Science
The Open University
Israel

The Gate-Cover Problem in Thin Polyominoes

A Thesis submitted as partial fulfillment of the requirements
towards an M.Sc. degree in Computer Science
The Open University of Israel
Department of Mathematics and Computer Science

By
Ariel Rosenberg

Prepared under the supervision of **Dr. Omrit Filtser.**

June 28, 2026

Abstract

Motivated by applications in path monitoring within urban networks, we introduce and investigate a new variant of the classical Art Gallery Problem, namely the Gate-Cover Problem.

We extend the traditional guarding framework by focusing on the coverage of designated gates rather than the surveillance of the entire region. Formally, we study the problem of placing the minimum number of cameras in a thin polyomino such that a designated set of gates is covered. We investigate two main variants: unbounded visibility range and bounded visibility range.

We prove that both variants are NP-hard. In particular, we show that the unbounded case remains NP-hard even for polyominoes restricted to T-junctions. For the unbounded variant, we establish an equivalence to edge domination in grid intersection graphs and design a near-linear-time 2-approximation algorithm. For the bounded-visibility case, we prove NP-hardness for every range parameter $k \geq 1$ and develop a polynomial-time approximation scheme (PTAS) using local search.

Beyond the theoretical analysis, we examine greedy heuristics and demonstrate experimentally that, with appropriate tie-breaking strategies, they consistently achieve near-optimal performance across a wide variety of synthetic city instances.

Overall, we advance both the theoretical understanding and the algorithmic toolkit for guarding problems in computational geometry, and we contribute practical methods relevant to real-world applications in surveillance networks.

Acknowledgements

I am deeply grateful to Dr. Omrit Filtser from the Open University of Israel for her exceptional supervision, invaluable insight, and constant guidance. Her patience, support, and encouragement during these difficult times of war in Israel have been a true source of strength. This work owes much to her expertise, dedication, and inspiring mentorship, for which I am sincerely thankful.
I also wish to thank Prof. Esther M. Arkin, Prof. Alon Efrat, Prof. Stephen Kobourov, Prof. Jan Kratochvíl, and Prof. Joseph S. B. Mitchell. Their guidance, ideas, proofs, feedback, and encouragement have significantly enriched this research.

Thanks to my wife for endless support, to my children, my parents and all my loves.

Contents

Abstract **i**

Acknowledgements **ii**

1 Introduction **1**
1.1 Thesis Contributions 3

2 Background and previous work **4**
2.1 AGP in polyominoes 5
2.1.1 Guarding polyominoes 5
2.1.2 Guarding polyominoes under k-hop visibility 6
2.1.3 Art gallery problem with rook and queen vision. 7
2.1.4 On r-guarding thin polyominoes 7
2.2 More related work 9
2.2.1 Bounds on guarding problems for orthogonal polygons in the plane 10
2.2.2 Partitioning into $\leq$ 8-vertex pieces and a mobile-guard bound. . . 10
2.2.3 The dispersive art gallery problem. 11

3 The Gate Cover Problem **12**
3.1 The city model 12
3.2 Problem statement 15

4 Hardness results **17**
4.1 Unbounded vision 17
4.1.1 NP-Hardness 18
4.2 Bounded vision 23
4.2.1 NP-Hardness 23

5 Algorithmic results **29**
5.1 A 2-approximation algorithm 29
5.2 Thin polyominoes with bounded treewidth 30
5.3 A separator based exact algorithm 32
5.4 A PTAS for the k-bounded version 34

6 Greedy heuristics and experimental results **36**
6.1 Lower bounds for the basic greedy heuristic 36
6.2 Greedy heuristics for unbounded vision 38
6.3 Greedy heuristics for bounded vision 38
6.4 Cities (Types of thin polyominoes) 39

6.5 Unbounded vision results 41
6.6 Bounded vision results 44
6.7 Real City Case Study 47

7 Conclusions **48**

Appendix **50**

Appendix A: Detailed Experimental Results: Unbounded Vision **51**
A.1 Detailed results for the cases involving gates placed on junctions 51
A.2 Detailed results for the cases where N gates are distributed randomly . . 54

Appendix B: Detailed Experimental Results: Bounded Vision **57**
B.1 Detailed results for the cases involving $k = 5$ 57
B.2 Detailed results for the cases involving $k \in \{5, 7, 10, 13, 16\}$ 60

List of Figures

1.1 An illustration of a point p guarded by a point g inside P, while p' is not. 1
1.2 The real city streets turn into a thin polyomino. 2

2.1 Illustration of k-hop visibility in a polyomino. 6
2.2 In r-guarding (left), a boundary point may see gates located on more than two streets. In the present setting (right), a camera positioned at the center can see only the streets that contain it, and does not see the striped squares. 8

3.1 Tetris game parts. 13
3.2 (a) Thin polyomino with $m = 107$. (b) Thin polyomino and its dual graph. 14
3.3 Gates 1–8 are highlighted in brown, and the cameras, labeled A and B, are marked in green. Note that A lies only on a vertical street $S_v(A)$ and B lies on both a vertical $S_v(B)$ and an horizontal $S_h(B)$ streets. 15

4.1 A thin polyomino with one horizontal street and three vertical streets that create three types of junctions (in brown). The middle one is a T-junction. 17
4.2 (a) Streets numbering. (b) Correspond bipartite graph. 18
4.3 A planar bipartite graph and its corresponding CNF formula. 20
4.4 The variable gadget (marked in blue) and its connection to clause gadgets (marked in green) of clauses that contain this variable. The connecting edges are red. Matching edges that dominate all edges of the variable gadget are displayed in bold. The variant in the left corresponds to $\phi(x) = true$, the variant to the right corresponds to $\phi(x) = false$. 21
4.5 The construction of the graph $G(\Phi)$ for a formula $\Phi = \alpha \wedge \beta \wedge \gamma \wedge \delta$ with $\alpha = (x \vee \neg y)$, $\beta = (\neg x \vee y \vee z)$, $\gamma = (y \vee z)$ and $\delta = (x \vee \neg z)$. A planar non-crossing drawing of $G(\Phi)$ in the right is derived from a non-crossing drawing of the incidence graph G_Φ of Φ depicted in the left. 22
4.6 Local modifications of paths in G' used to enforce path lengths congruent to 1 modulo 3. Blue vertices represent vertices of degree 3, while red vertices correspond to vertices of G' inherited from bending points in the orthogonal drawing of G. 25
4.7 Propagation of dominating vertices along the path P_{uv}. Vertices in the dominating set are shown in blue, with arrows indicating the vertices they dominate. 26
4.8 The variable gadget (left) and a reduced core used to adjust port coordinates (right). 27
4.9 The linking gadget (left) and the propagation of a false value along it (right). 28

5.1 (a) A polyomino with 3 horizontal and 3 vertical streets. An optimal solution has size two, for example, using the cameras marked in green diamonds. (b) The graph $\mathcal{G}^{\mathcal{X}}$. H_3 and V_3 are dummy streets. Optimal vertex cover has size 4. 30
5.2 Reduction of the dual graph while preserving bounded treewidth. 31
5.3 The three possible separator cases. 33

6.1 Left: a solution produced by the greedy algorithm. Right: an optimal solution. 37
6.2 A counterexample family for the greedy algorithm, parameterized by $k = 4$. 37
6.3 (a) An optimal solution for bounded visibility with $k = 5$. (b) Candidate camera locations considered for obtaining the optimal solution. 39
6.4 Types of cities and gates spreading. 40
6.5 Real city 47

List of Tables

6.1 Results for cities with gates at junctions 43
6.2 Results for cities with randomly distributed gates 43
6.3 Results for bounded visibility with $k = 5$ 46
6.4 Results for varying visibility radius k 46

1 Introduction

The *Art Gallery Problem* (AGP), introduced by Victor Klee in 1973, is one of the classical and most influential problems in computational geometry. It is motivated by the following question: "In an art gallery, what is the minimum number of guards who together can observe the whole gallery?" [40]. The Problem is formally defined as follows: Given a polygon P, the goal is to determine the minimum number of guards whose placement guarantees that every point of P is visible to at least one guard. A point $p \in P$ is said to be guarded by a point $g \in P$ if and only if the line segment $\overline{gp}$ lies entirely within P (see Figure 1.1).

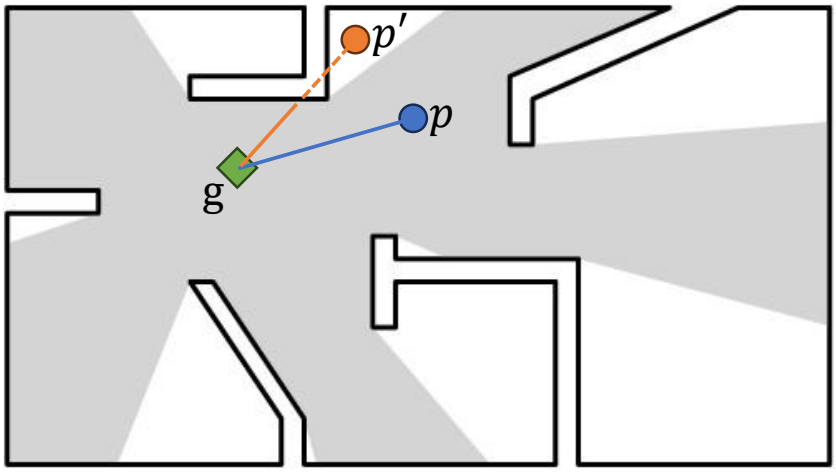


Figure 1.1: An illustration of a point p guarded by a point g inside P, while p' is not.

Chvátal's theorem established that $\lfloor n/3 \rfloor$ guards are always sufficient, and sometimes necessary, to guard a simple polygon with n vertices, where a simple polygon is a planar polygon whose boundary does not intersect itself. For the important special case of orthogonal polygons, that is, polygons whose edges are all axis-parallel, the corresponding bound improves to $\lfloor n/4 \rfloor$ [12].

Since its introduction, AGP has motivated an extensive body of research, with many variants studied under diverse vision models, polygonal classes, and algorithmic frameworks. Variants have been proven to be NP-hard or APX-hard, making them central testbeds for approximation algorithms and complexity analysis.

Over the years, many variants of the Art Gallery Problem have been studied. These variants differ in several natural ways. the guards may be restricted to vertices, to the boundary, or allowed anywhere in the domain. the domain itself may be a simple polygon, an orthogonal polygon, a weakly visible polygon, or a polyomino. The notion of visibility may vary, for example line-of-sight visibility, r-visibility, pixel-based visibility, or other restricted models. Many of these variants are computationally hard, while for

important special cases there are approximation algorithms, polynomial-time algorithms, or structural theorems [4–7, 24, 29, 35].

The problem studied in this thesis belongs to this broader landscape, but differs from the classical formulations in two essential respects. First, the objective is not to guard the entire domain, but only a designated subset of locations, called gates. Second, the visibility model is defined by movement along horizontal and vertical streets in a thin polyomino. These restrictions lead to a different combinatorial structure from the one arising in the classical art gallery setting, and therefore require separate analysis.

Despite its elegance, guarding the entire polygon is often unnecessary in realistic applications. In many scenarios, it suffices to track or delimit intruders relative to a reduced set of critical points - such as entry gates, intersections, or communication hubs. For example, consider a city in which a thief moves along the streets, and the objective is to confine his location to a specific street, even without knowing his exact position. This idea motivates the study of the *Gate-Cover Problem*, a novel variant of AGP introduced in this thesis. A thin polyomino is used to represent the city. A polyomino P is a polygon formed by joining together $|P| = m$ unit squares on the square lattice, and a polyomino is thin if it does not contain a 2×2 block of unit squares. In this analogy, a "street" would be a maximal sequence of unit squares in P such that every pair of consecutive unit squares shares a horizontal (resp. vertical) edge. Instead of requiring visibility of all points in the thin polyomino, the objective is to guard a subset X of designated unit squares within a thin polyomino P, referred to as gates. Cameras are positioned at the centers of unit squares, and a gate is covered if it lies on the same horizontal or vertical street as a camera (see Figure 1.2). This model is motivated by the problem of monitoring road networks: a city is represented as a thin polyomino, gates correspond to junctions or critical cells, and cameras monitor along entire streets. The motivation connects directly to emerging architectures in vehicle-to-vehicle (V2V) communication and smart transportation systems, where minimizing sensors is crucial for efficiency and cost [20, 44].

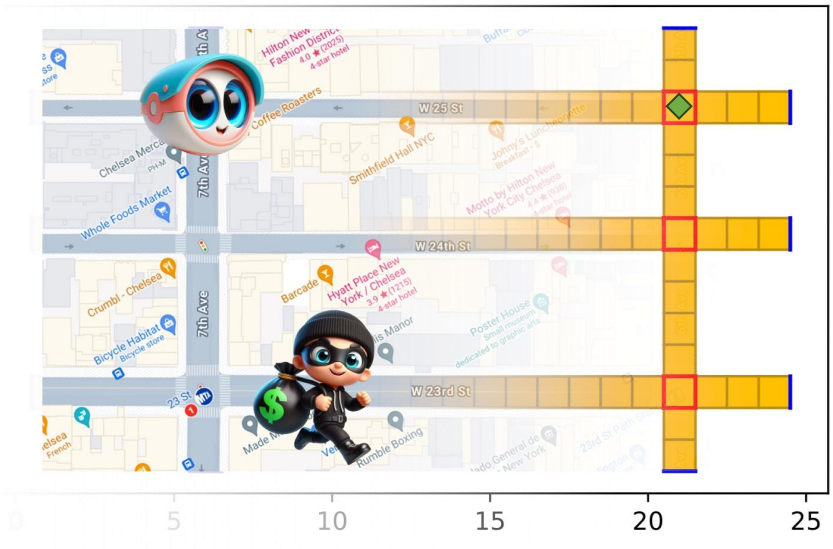


Figure 1.2: The real city streets turn into a thin polyomino.

1.1 Thesis Contributions

The main contributions of this thesis are as follows:

- **Problem Definition.** The Gate-Cover Problem in thin polyominoes is formally introduced as the task of covering a specified set of gates by placing cameras on unit squares, where visibility is restricted to horizontal and vertical streets.
- **Hardness Results.** Both the unbounded-range and bounded-range variants are shown to be NP-hard. In the unbounded case, hardness holds even for highly restricted thin polyominoes consisting solely of T-junctions. For bounded range k, NP-hardness is established for every $k \geq 1$ via reductions from planar domination and planar satisfiability problems.
- **Approximation Algorithms.** For the unbounded case, a near-linear time 2-approximation algorithm is developed via a reduction to edge domination in bipartite grid intersection graphs. For bounded range k, the problem is shown to admit a polynomial-time approximation scheme (PTAS) for constant k, using the local-search framework for geometric covering.
- **Greedy Heuristics and Experimental Evaluation.** Extensive computational experiments are conducted on synthetic city instances modeled as thin polyominoes. Several greedy heuristics are evaluated, and refined tie-breaking strategies consistently produce solutions significantly closer to optimal than worst-case guarantees suggest, often achieving approximation ratios below 1.5.

  Taken together, these contributions position the Gate-Cover Problem in thin polyominoes as a rich and practically motivated extension of geometric guarding and domination, with deep connections to graph theory, approximation algorithms, and real-world monitoring applications.

  The novelty of the problem formulation, together with the proofs and results presented in this thesis, was recognized and accepted for publication at **WALCOM 2026** (Workshop on Algorithms and Computation), an international conference devoted to advances in algorithms and computational theory [2].

2 Background and previous work

There is a large body of work on the many variants of the Art Gallery Problem (AGP), including numerous studies on guarding orthogonal polygons and polyominoes under different visibility models. Since the literature on the AGP is vast, only the most relevant works are discussed here. For comprehensive surveys and detailed background on the problem and its variants, see, for example, the monograph by O'Rourke [40] and the survey chapter by Urrutia [45].

The classical Art Gallery Problem (AGP), introduced by Klee in 1973 and first solved by Chvátal, asks for the minimum number of guards required to observe an entire simple polygon. Chvátal's Art Gallery Theorem states that $\lfloor n/3 \rfloor$ guards are always sufficient and sometimes necessary to guard any simple polygon with n vertices [12]. This bound is tight, and Fisk later provided a short and elegant proof based on triangulations and 3-colorings of planar graphs [23]. The problem initiated a vast line of research in computational geometry and combinatorial optimization.

From a computational perspective, determining the minimum number of guards needed to guard a given polygon is NP-hard. Hardness results hold for various guard models, including point guards and vertex guards, and even under strong geometric restrictions [30, 32]. Several versions of the problem are APX-hard, ruling out polynomial-time approximation schemes unless P = NP [19]. In addition, variants involving edge guards, sliding cameras, and visibility with constraints on angles or distances have been shown to exhibit similar computational intractability [5, 8, 15, 32].

Despite these hardness results, significant progress has been made on approximation algorithms and special cases. Constant-factor approximation algorithms are known for several guard models, and polynomial-time algorithms exist for restricted polygon classes such as monotone polygons and certain subclasses of orthogonal polygons [4, 24, 38]. Numerous extensions and variants have been studied, including guarding polygons with holes, terrains, polyominoes, and grid-based structures, as well as problems related to domination and covering in planar and geometric graphs [6, 13, 16, 22, 24, 37].

The following sections review the prior work most relevant to the present thesis. The discussion begins with art gallery variants on polyominoes, since thin polyominoes form the underlying domain of the Gate-Cover Problem and several of the visibility models considered there are closely related to the one studied here. It then turns to orthogonal polygons and related guarding models whose structural and algorithmic ideas, such as rectangular visibility, bounded treewidth, and graph-based discretization, strongly influence the methods used later in this thesis.

2.1 AGP in polyominoes

A *Polyomino* is a finite union of unit axis-aligned squares in the plane whose interiors form a connected set under edge adjacency. A *Thin polyomino* is a polyomino that contains no 2×2 block of unit squares. Since thin polyominoes play a central role in this work, several important results from prior studies on art gallery problem variants in polyominoes are reviewed.

2.1.1 Guarding polyominoes

Biedl et al. [6] study the art gallery problem in the special setting where the gallery is an *m-polyomino*, namely a connected union of m unit axis-aligned squares (pixels), whose interior is connected. In contrast to the classical parametrization by the number of polygon vertices, the paper investigates guarding complexity as a function of m, the number of pixels.

The authors consider two types of guards. *Point guards* are placed at arbitrary points in the polyomino, while *pixel guards* occupy entire pixels.

Visibility is defined in several models:

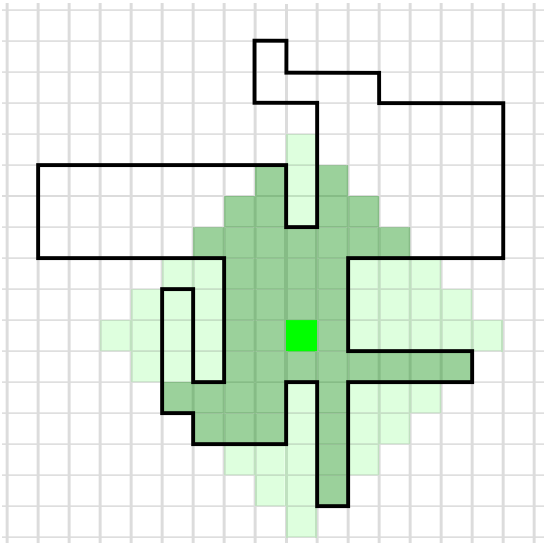
- **Unrestricted visibility.** Two points $a, b \in P$ see each other if the entire line segment ab is contained in P. This is the classical art-gallery visibility model, allowing guards to see in all directions without geometric restriction beyond line-of-sight containment. A pixel (or region) is considered guarded if every point within it is visible from at least one guard location. This model is the most permissive among the three and therefore admits the largest set of feasible guard placements.
- **All-or-nothing visibility.** A pixel p is guarded only if there exists a single guard location a such that *all* points $b \in p$ satisfy $ab \subseteq P$. Partial visibility of a pixel is insufficient; complete coverage from one position is required. This model is strictly more restrictive than unrestricted visibility, since guarding must occur simultaneously for the entire pixel. Any cover valid under r-visibility is also valid here, but not conversely.
- **r-visibility.** Two points $a, b \in P$ see each other if the axis-parallel rectangle spanned by a and b is entirely contained in P. Visibility is therefore constrained to orthogonal, rectangle-based sight, rather than arbitrary straight-line segments. This model is the most restrictive of the three, as it imposes a structural alignment condition in addition to containment. Every r-visibility cover is an all-or-nothing cover, and hence also an unrestricted cover, but the reverse implications do not hold in general.

The main combinatorial contribution establishes tight bounds on the maximum number of guards required in terms of m. For point guards, they prove that $\lfloor \frac{m+1}{3} \rfloor$ guards are always sufficient and sometimes necessary to guard any m-polyomino, even in the r-visibility model. For pixel guards, they show that $\lfloor \frac{3m}{11} \rfloor + 1$ pixel guards are always sufficient and sometimes necessary. Both bounds apply to polyominoes with or without holes. From an algorithmic perspective, the paper proves that computing an optimal guard set for a given m-polyomino is NP-hard. The hardness result holds for both point guards and pixel guards, and already applies to simple polyominoes, even under the all-or-nothing visibility model. The proof is by a reduction from MAX2SAT(2L), via an intermediate reduction to a line covering problem and a geometric construction that

forces guards to correspond to selected literals. Despite this hardness, the authors identify tractable special cases. For thin polyominoes with dual graph to be tree, they show that optimal guard placements can be computed in polynomial time in the r visibility and all-or-nothing models. In the even more restricted case of thin polyomino paths, they give simple greedy algorithms that run in linear time and are provably optimal. These results demonstrate that structural restrictions on the polyomino lead to efficient exact algorithms, despite the general NP-hardness of the problem.

2.1.2 Guarding polyominoes under k-hop visibility

Filtser et al. [22] introduce the *Minimum k-Hop Guarding Problem* (MkGP) in polyominoes, a variant of the art gallery problem motivated by coverage under bounded walking distance. In this model, a guard placed in a unit square s sees another square t if the distance between s and t in the dual graph of the polyomino is at most k, That is, visibility is defined by graph distance rather than straight-line sight. Equivalently, guards cover all squares within k hops in the adjacency graph of cells, allowing visibility "around corners" (Figure 2.1).



A unit square in green with its k-hop visibility region for $k = 6$ (shaded in dark green) within a polyomino. The light green diamond indicates the maximal reachable region under L_1 distance (This picture was taken from [22]).

Figure 2.1: Illustration of k-hop visibility in a polyomino.

The authors first analyze the combinatorial complexity of the visibility model. They show that the VC-dimension of k-hop visibility is exactly 3 in simple polyominoes and 4 in polyominoes with holes, establishing tight bounds. They then study computational complexity and prove that MkGP is NP-complete for all $k \geq 2$, even when restricted to thin polyominoes with holes. The hardness proof is obtained by a reduction from Planar Monotone 3SAT, using variable, clause, and connector gadgets that propagate assignments via forced guard placements while respecting k-hop distance constraints.

On the algorithmic side, the paper identifies a tractable subclass. A polyomino is called 2-*thin* if it does not contain a 3×3 block of unit squares. such polyominoes have constant treewidth. For simple 2-thin polyominoes, the authors present a linear-time 4-approximation algorithm for MkGP, valid for all $k \in \mathbb{N}$. The algorithm constructs a tree structure (a skeleton graph) over the polyomino and places guards iteratively, adding at most four guards per step while maintaining a set of witness squares whose mutual

distances exceed k. Since the witness set forms a lower bound on the optimal solution, this yields a factor-4 approximation.

2.1.3 Art gallery problem with rook and queen vision.

Alpert and Roldán [1] study a discrete variant of the art gallery problem in which the gallery is a polyomino and guards are chess pieces with restricted lines of sight. They introduce two visibility models inspired by chess: *rook vision* and *queen vision*. A rook placed on a tile guards all tiles in the same row or column, as long as the entire line segment between tile centers is contained in the polyomino. A queen extends this visibility by additionally guarding tiles along diagonal directions, again restricted to paths fully contained within the polyomino. Thus, visibility is combinatorial and grid-based, rather than continuous.

The paper establishes tight worst-case bounds on the number of guards needed. For any polyomino with n tiles, the authors prove that $\lfloor \frac{n}{2} \rfloor$ rooks are always sufficient and sometimes necessary to guard all tiles under rook vision, and that $\lfloor \frac{n}{3} \rfloor$ queens are always sufficient and sometimes necessary under queen vision. Both bounds are shown to be tight by explicit constructions, and they extend to d-dimensional polycubes with the same asymptotic ratios. From a computational perspective, the paper proves that determining the minimum number of rooks or queens required to guard a given polyomino is NP-hard. The proof is by a polynomial-time reduction from Planar 3SAT. Each variable and clause is encoded by a carefully designed polyomino gadget, connected by narrow corridors that propagate visibility constraints. The construction ensures that feasible guard placements correspond exactly to satisfying truth assignments, while the number of guards is fixed. The NP-hardness holds for both rook vision and queen vision, and already applies in two dimensions.

2.1.4 On r-guarding thin polyominoes

Biedl and Mehrabi study the r-guarding problem in orthogonal polygons, with particular emphasis on thin polygons [7]. In their model, visibility is defined via *axis-aligned rectangular visibility*, commonly referred to as r-visibility. Given two points $g, p \in P$, the guard g is said to r-guard p if the minimum axis-aligned rectangle

$$R(g,p) = [\min(x_g, x_p), \max(x_g, x_p)] \times [\min(y_g, y_p), \max(y_g, y_p)]$$

is fully contained in the polygon P. Intuitively, this means that g can see p if one can move from g to p using only horizontal and vertical directions without ever leaving the polygon. Unlike classical line-of-sight visibility, the straight segment gp is not relevant. Instead, the entire bounding rectangle must lie inside P. This model is symmetric and naturally adapted to orthogonal geometry. The notion of r-visibility can be understood geometrically as follows. For a fixed guard g, the set of points that it can r-guard forms an orthogonal region composed of maximal axis-aligned rectangles containing g. A guard may therefore see points across multiple "streets" of the polygon, provided the corresponding spanning rectangles remain inside P. This behavior is illustrated on the left side of Figure 2.2, where a boundary guard may simultaneously r-guard gates located along more than two orthogonal directions.

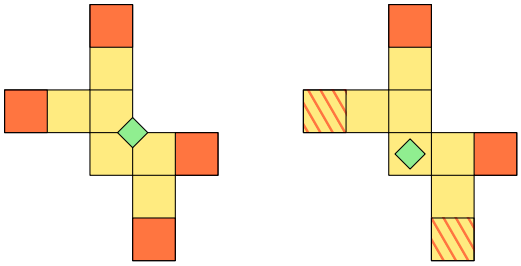


Figure 2.2: In r-guarding (left), a boundary point may see gates located on more than two streets. In the present setting (right), a camera positioned at the center can see only the streets that contain it, and does not see the striped squares.

The paper resolves several central complexity questions. It proves that r-guarding is NP-hard on thin orthogonal polygons with holes. The reduction is obtained from planar Vertex Cover, by constructing a thin polygon whose maximal rectangles encode edges of the graph. Thus, even under the structural restriction of thinness, the presence of holes suffices to make the problem computationally intractable. At the same time, the paper shows that if the polygon is simple and thin (equivalently, its dual graph is a tree), then the problem becomes tractable: an optimal solution can be computed in linear time. More generally, if the dual graph of the polygon has bounded treewidth, the problem is solvable in linear time via a reduction to a restricted distance-2-dominating set problem and an application of Courcelle's theorem [14]. Furthermore, the framework allows flexibility in specifying the region $U \subseteq P$ to be guarded and the set Γ of admissible guards, and yields fixed-parameter tractability with respect to the number of holes and a suitable notion of thickness. Overall, the paper gives a clear picture of the computational complexity of r-guarding in thin orthogonal polygons. It shows that the problem becomes NP-hard when the polygon contains holes, while it can be solved in linear time when the polygon is simple and thin (that is, when its dual graph is a tree). These results identify the structural properties that determine whether the problem is computationally tractable or intractable under the r-visibility model.

2.2 More related work

AGP in orthogonal polygons. The Art Gallery Problem has been studied in many variants, differing in the type of domain, the allowed guard locations, and the underlying visibility model. Orthogonal polygon is a polygon whose edges are all horizontal or vertical. Orthogonal polygons constitute one of the most important and extensively studied special cases of the Art Gallery Problem. In these polygons, all edges are axis-aligned, a restriction that naturally models many realistic environments such as building floor plans, urban street layouts, and grid-based infrastructures. The orthogonal structure simplifies geometric reasoning and often allows combinatorial techniques to be applied more effectively than in the general case. At the same time, orthogonal polygons retain substantial expressive power and capture many practical guarding scenarios, making them both theoretically appealing and practically relevant. It was proved multiple times [27, 34, 39] that $\lfloor \frac{n}{4} \rfloor$ point guards are always sufficient and sometimes necessary to guard the interior of a simple orthogonal polygon with n vertices. Finding the minimum number of guards is NP-hard on simple polygons [32], even on simple orthogonal polygons [43].

Guarding with sliding cameras. Biedl et al. [5], building on earlier work ([29, 35]), study the problem of guarding orthogonal polygons using *sliding cameras*, that is, guards moving along horizontal or vertical segments and seeing points by perpendicular projection onto their trajectory.

Their central contribution is a geometric discretization via *pixelation*, which reduces the continuous visibility problem to a finite combinatorial structure. This reduction translates guarding into a hitting-set formulation and further into a distance-2 domination problem on an auxiliary graph derived from slice segments and pixel crosses. Using this framework, they obtain the first constant-factor approximation for the Minimum Sliding Camera (MSC) problem, prove NP-hardness for MSC and its horizontal-only variant, and show linear-time solvability when the pixel dual graph has bounded treewidth (in particular, for thin hole-free polygons).

They prove that the Minimum Sliding Camera (MSC) problem admits a constant-factor approximation, even for orthogonal polygons with holes, via a tree-decomposition framework. Moreover, if the dual graph of the pixelation has bounded treewidth, MSC can be solved in polynomial time. In particular, it is solvable in linear time for thin hole-free polygons. The same structural approach applies to the horizontal-only variant (MHSC). Both MSC and MHSC are shown to be NP-hard. In addition, sliding-camera analogues of art gallery theorems are established. For an orthogonal polygon with n vertices, $\lfloor (3n + 4)/16 \rfloor$ sliding cameras are always sufficient and sometimes necessary. If the dual graph of the vertical decomposition is a path, $\lfloor (n + 2)/6 \rfloor$ sliding cameras suffice. For horizontal sliding cameras, $\lfloor n/4 \rfloor$ guards are always sufficient and sometimes necessary.

The discretization and structural graph-theoretic approach introduced by Biedl et al. are particularly relevant, as they strongly influence the techniques developed for the Gate-Cover Problem studied in this thesis.

2.2.1 Bounds on guarding problems for orthogonal polygons in the plane

Daescu and Malik [15] investigate guarding problems in orthogonal polygons with holes under a restrictive visibility model in which each guard is a *vertex guard* with a bounded field of view of angle 180°. The central feature of this model is the limitation imposed by the guard's viewing angle: visibility is confined to a single halfplane, rather than the full 360° visibility typically assumed in classical formulations. Guards are placed exclusively at vertices of the holes, and each guard is oriented so that its visible and invisible regions are separated by a line parallel to one of the edges incident to that vertex. Thus, the orientation of the 180° viewing cone is constrained by the local geometry of the polygon. The objective is to determine a placement and orientation of such halfplane guards that collectively cover both the free space of the polygon and its boundary.

Axis-aligned orthogonal monotone holes. In the first setting, the holes are orthogonal, axis-aligned, and monotone with respect to both the x- and y-axes. Let P be an axis-aligned rectangle containing k disjoint monotone orthogonal holes with a total of m vertices. The authors use a geometric decomposition based on extending edges of the holes to partition the free space into monotone staircase regions. Each such staircase can be fully guarded by a single vertex guard placed at an appropriate corner and oriented along one of the coordinate directions.

By carefully analyzing the number of staircase regions created by this process and exploiting structural properties of reflex and convex vertices of orthogonal polygons, the authors show that $\frac{m}{2} + \lfloor \frac{k}{4} \rfloor + 4$ vertex guards are always sufficient to guard the free space. This bound holds when all guards are placed on vertices of the holes and is independent of the arrangement of the holes inside the rectangle.

Arbitrary-oriented orthogonal holes. In the second setting, the orthogonal holes are no longer required to be axis-aligned, although the outer polygon remains an axis-aligned rectangle. The same model of vertex guards with 180° visibility is used. In this more general case, the authors construct a family of instances in which the relative orientation of the holes severely restricts visibility between them.

Using a detailed lower-bound construction based on counting reflex vertices and analyzing regions that can only be seen from very limited guard positions, the authors prove that $\frac{m}{2} + k + 1$ vertex guards are sometimes necessary to guard the free space. This result establishes a strict separation between the axis-aligned and arbitrarily oriented cases. The authors conjecture that this bound is tight, but no matching sufficiency result is currently known.

2.2.2 Partitioning into $\leq$ 8-vertex pieces and a mobile-guard bound.

Györi et al. [25] proves a sharp *partition theorem* for simple (hole-free) orthogonal polygons (also called simple polyominoes). Let P be a simple polyomino with n vertices. The main result shows that P can be partitioned into at most $\lfloor \frac{3n+4}{16} \rfloor$ simple polyominoes, each having at most 8 vertices. Since any $\leq$ 8-vertex polyomino is guardable by a single *mobile guard* (a guard patrolling a line segment and seeing points by line-of-sight within

the polygon), this yields a new and shorter proof that $\lfloor \frac{3n+4}{16} \rfloor$ mobile guards always suffice to guard the interior of any n-vertex simple orthogonal polygon.

2.2.3 The dispersive art gallery problem.

Rieck and Scheffer [41] introduce different variant of the classical art gallery problem, motivated by safety and robustness considerations. Instead of minimizing the number of guards, the objective is to maximize the *minimum pairwise distance* between guards. Formally, given a polygon P and a real value ℓ, the *Dispersive Art Gallery Problem* asks whether there exists a guard set G that covers P such that the geodesic L_1-distance between every pair of guards in G is at least ℓ. The paper studies this problem for *vertex guards* in polyominoes under rectangular visibility.

The authors first establish tight worst-case bounds for simple (hole-free) polyominoes. They construct thin polyominoes in which every feasible guard set has dispersion distance at most 3, and show that this bound is worst-case optimal. More precisely, they prove that for every simple polyomino there exists a guard set with dispersion distance at least 3, and they give a constructive polynomial-time algorithm that computes such a guard set. Together, these results imply that the maximum achievable dispersion distance in simple polyominoes is exactly 3.

The paper then studies the computational complexity of maximizing the dispersion distance. It is shown that deciding whether a dispersion distance of at least 5 can be achieved is NP-complete, even when restricting to thin polyominoes with holes. The proof is by a reduction from Planar Monotone 3SAT, using carefully designed geometric gadgets that encode variable assignments and clause satisfaction while enforcing distance constraints between guards.

Finally, the authors identify a tractable subclass. For tree-shaped polyominoes, whose dual graph is a tree, they present a dynamic programming algorithm that computes a guard set maximizing the minimum pairwise distance. This algorithm runs in linear time and yields optimal solutions.

3 The Gate Cover Problem

Consider a city in which a thief moves freely along the streets, while the authorities do not know his exact position at any given time. The goal is not necessarily to pinpoint his precise location, but rather to guarantee that he can be confined to, or detected within, a specific street or a small region of the city. By strategically placing observation points, such as cameras or patrol units, one aims to ensure that regardless of the thief's movements, his presence can always be inferred within a limited area.

This scenario is closely related to the classical art gallery problem. In the traditional art gallery problem, the objective is to place guards so that every point of a region is directly visible. In contrast, in this variant the goal is weaker but more robust, instead of full visibility, the task is to reduce uncertainty about an intruder's location. Guards are placed so that, even without continuous line-of-sight to the intruder, their combined coverage guarantees that the intruder cannot move without being constrained to a known subset of the environment.

From a theoretical perspective, this leads to art gallery formulations that focus on localization rather than visibility. The emphasis shifts from seeing every point to ensuring that every possible position of the intruder is "covered" in an informational sense. This change results in new combinatorial and algorithmic questions, distinct from those studied in classical guarding problems, and gives rise to new problem variants with different complexity and structural properties.

The connection to real-world applications is direct. Urban surveillance systems, sensor networks, and autonomous patrol planning often operate under uncertainty, where exact positions cannot be continuously tracked due to limited sensing, visual obstacles, or resource constraints. In such settings, it is often sufficient to narrow down the location of an intruder or event to a small region rather than to know it precisely. Theoretical results from these art gallery variants provide principled guidelines for placing sensors or cameras so as to minimize uncertainty, offering insight into how abstract geometric problems can inform practical security and monitoring strategies in complex environments.

3.1 The city model

We introduce the concept of polyominoes and thin polyominoes. Let $\mathbb{Z}^2$ denote the integer lattice in the plane. For each $(i, j) \in \mathbb{Z}^2$, define the unit square

$$Q_{i,j} = [i, i+1] \times [j, j+1] \subset \mathbb{R}^2.$$

Definition 1. *Polyomino*
A polyomino *is a set*

$$P = \bigcup_{(i,j)\in S} Q_{i,j},$$

where $S \subset \mathbb{Z}^2$ is a finite set of lattice points such that the adjacency graph

$$G_S = (S, E)$$

is connected, where

$$\{(i,j),(i',j')\} \in E \quad \textit{if and only if} \quad |i-i'| + |j-j'| = 1.$$

That is, connectivity is defined with respect to shared edges (4-connectivity), and not merely shared vertices.

According to this definition, a polyomino is a finite union of unit axis-aligned squares in the plane whose interiors form a connected set under edge adjacency. In particular, connectivity is defined with respect to shared edges (4-adjacency) and not merely shared vertices.

The most well-known family of polyominoes is given by the shapes used in the game *Tetris* (Figure 3.1).

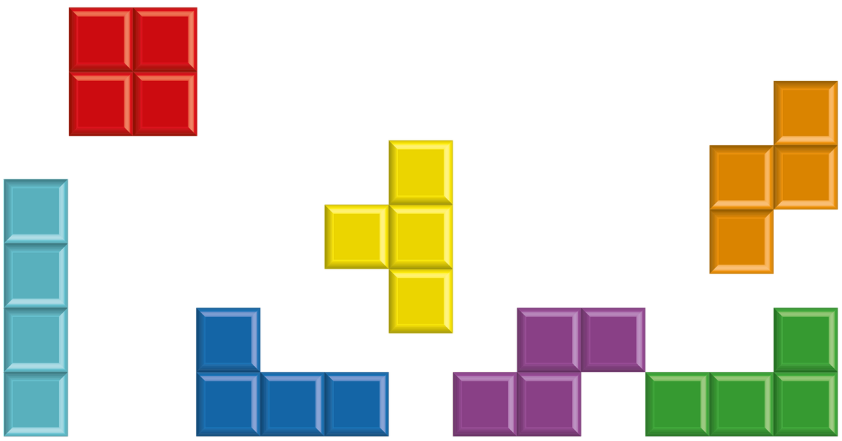


Figure 3.1: Tetris game parts.

Definition 2 (Thin Polyomino)**.** *Let*

$$P = \bigcup_{(i,j)\in S} Q_{i,j}$$

be a polyomino, where

$$Q_{i,j} = [i, i+1] \times [j, j+1]$$

and $S \subset \mathbb{Z}^2$ is finite and 4-connected. Recalling $V = \mathbb{Z}^2$ denote the set of lattice vertices, and let $\mathrm{int}(P)$ *denote the topological interior of P, that is, the set of all points $x \in P$ for which there exists $\varepsilon > 0$ such that the open ball $B(x, \varepsilon) \subseteq P$. The polyomino P is called* thin *if for every $v \in V$,*

$$v \notin \mathrm{int}(P).$$

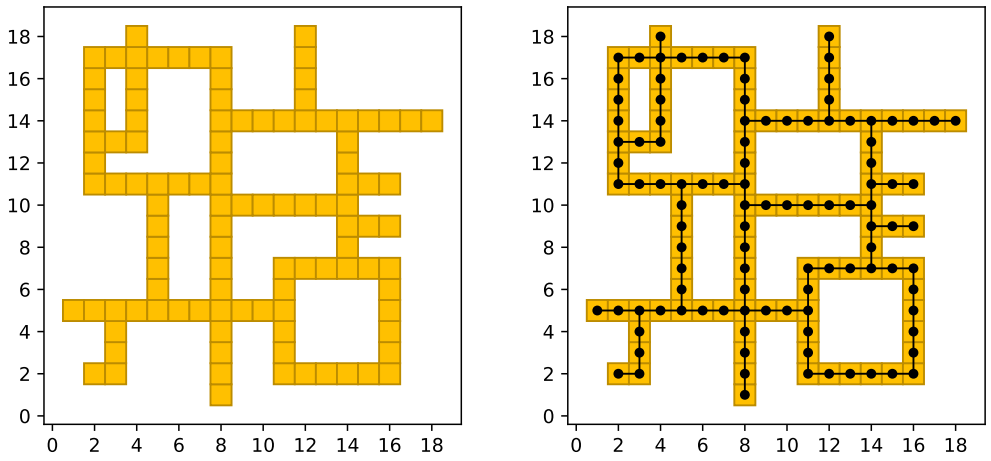
Equivalently, no lattice vertex $v \in \mathbb{Z}^2$ lies strictly in the interior of P.

According to this definition, a thin polyomino is a polyomino that contains no 2×2 block of unit squares.

The city is modeled by representing its street layout as a thin polyomino P. Each unit square of P corresponds to a small city block or intersection area, and adjacent squares represent blocks that are directly connected by a street segment. The thinness assumption captures the structure of typical urban street networks, in which streets form narrow corridors and wide open square areas are limited. Under this model, maximal horizontal and vertical streets of the polyomino correspond to straight city streets, while intersections arise at their crossings. This abstraction allows us to reason about movement, visibility, and monitoring in the city using a discrete geometric framework, while preserving the essential connectivity and directional constraints of real-world street layouts.

Let P be a *thin polyomino*, that is, a polyomino that does not contain a 2×2 block of unit squares. Equivalently, P is a connected union of unit squares whose interior contains no grid vertex of degree four. The number of unit squares of P is denoted by m (Figure 3.2a). Thin polyominoes form an important and well-studied subclass of polyominoes, as their geometry imposes strong structural restrictions while still allowing nontrivial combinatorial behavior.

The structure of P is captured by its *dual graph* G_P. The graph G_P has one vertex for each unit square of P, placed at the center of that square, and an edge between two vertices whenever the corresponding unit squares share a common edge. Since each square has at most four neighbors, every vertex of G_P has degree at most four. Because P is thin, the graph G_P does not contain any 2×2 grid subgraph (Figure 3.2b).



(a) (b)

Figure 3.2: (a) Thin polyomino with $m = 107$. (b) Thin polyomino and its dual graph.

A *vertical street* (resp. *horizontal street*) is defined as a maximal path in G_P consisting only of vertical (resp. horizontal) edges. Intuitively, streets correspond to maximal straight corridors of adjacent squares in P. Each street can be represented as a vertical or horizontal line segment in the plane whose endpoints have integer coordinates. Two streets with the same orientation are always disjoint, while a vertical and a horizontal street may intersect in at most one grid point or touch at an endpoint.

For a vertex $u \in V(G_P)$, The horizontal street containing u is denoted by $S_h(u)$, and the vertical street containing u is denoted by $S_v(u)$. If no such street exists, the corresponding value is set to NULL. Note that, due to thinness, a vertex belongs to at most one horizontal and at most one vertical street, which simplifies the visibility structure significantly.

3.2 Problem statement

Let $\mathcal{X} \subseteq V(G_P)$ be a distinguished subset of vertices, called *gates*. A camera placed at a vertex $u \in V(G_P)$ is said to *see* a gate $x \in \mathcal{X}$ if $x \in S_h(u)$ or $x \in S_v(u)$, that is, if u and x lie on the same horizontal or vertical street. This visibility model reflects line-of-sight along straight corridors of the polyomino.

For example, in Figure 3.3, camera A sees only gate 8, which lies on its vertical street $S_v(A)$. In contrast, camera B sees gates 1–3 along its horizontal street $S_h(B)$, and gates 4–7 along its vertical street $S_v(B)$.

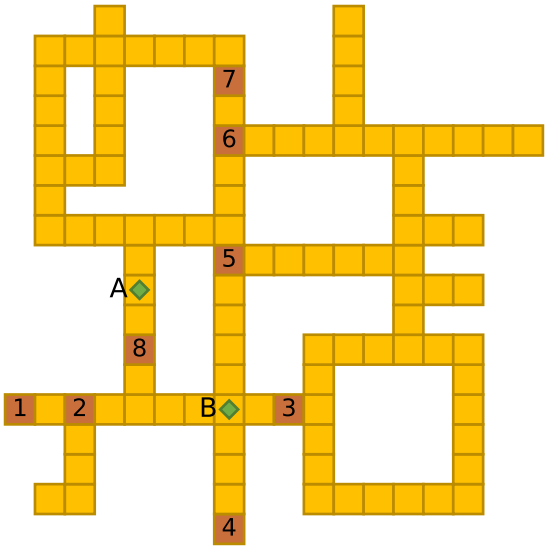


Figure 3.3: Gates 1–8 are highlighted in brown, and the cameras, labeled A and B, are marked in green. Note that A lies only on a vertical street $S_v(A)$ and B lies on both a vertical $S_v(B)$ and an horizontal $S_h(B)$ streets.

The central computational problem studied in this work is defined as follows.

The gate-cover problem

Given a thin polyomino P and a set of gates $\mathcal{X}$, find a minimum-size set $C \subseteq V(G_P)$ of camera locations such that every gate in $\mathcal{X}$ is seen by at least one camera in C.

It is possible to switch freely between the graph-based representation of a thin polyomino and its geometric interpretation as a collection of horizontal and vertical streets embedded in the integer grid. Under this interpretation, both gates and cameras are points with integer coordinates, and visibility corresponds to alignment along a common grid segment. This dual view allows us to combine combinatorial graph techniques with geometric reasoning.

A bounded-range variant of the model is also considered. Given an integer parameter $k \geq 0$, a camera placed at u sees a gate x if u and x lie on the same street and the graph distance between them in G_P is at most k. In this case, visibility is restricted both by direction and by distance, and depends on the hop-distance along the street.

Another case of computational problem studied in this work is defined as follows.

The bounded gate-cover problem

Given a thin polyomino P, a set of gates $\mathcal{X}$, and an integer parameter $k \geq 1$, find a minimum-size set $C \subseteq V(G_P)$ of camera locations such that every gate in $\mathcal{X}$ is seen by at least one camera in C, where a camera at u sees a gate x if u and x lie on the same street and their graph distance in G_P is at most k.

4 Hardness results

In this chapter, we show that the Gate-Cover Problem is NP-hard in both visibility settings considered in this thesis: the unbounded-visibility model and the bounded-visibility model.

4.1 Unbounded vision

In this section we consider the case in which cameras have an unbounded range of vision, that is, a camera placed at u guards any gate x in $S_h(u)$ or $S_v(u)$. Clearly, a gate x is guarded if and only if a camera is placed on either $S_h(x)$ or $S_v(x)$.

Denote by H (resp. V) the set of horizontal (resp. vertical) streets of P. Let $\mathcal{J}$ be the set of "junctions" (Figure 4.1): all the vertices of G_P that correspond to intersection points between two streets. Observe that in the case of unbounded vision, it may be assumed that all cameras are placed on junctions. If a camera is not placed on a junction, it can be moved to the nearest junction on the same street without affecting feasibility (the problem is trivial if no such junction exists).

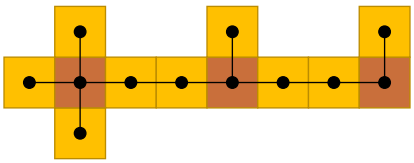
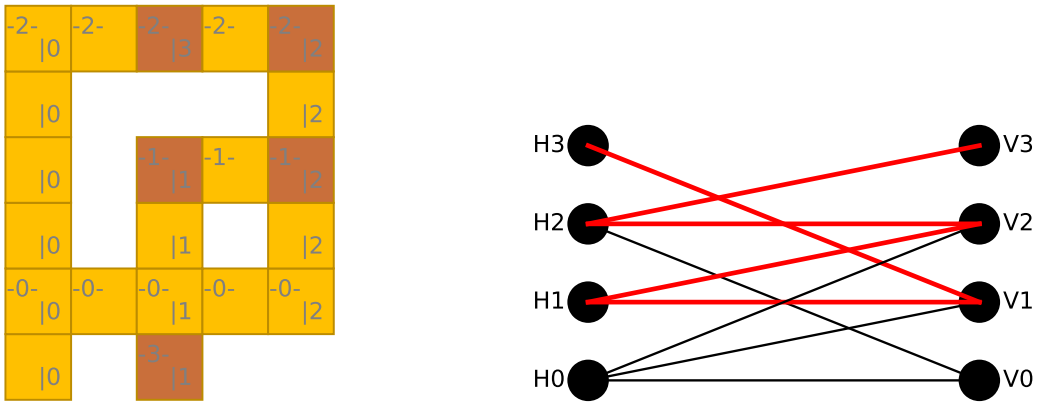


Figure 4.1: A thin polyomino with one horizontal street and three vertical streets that create three types of junctions (in brown). The middle one is a T-junction.

Consider the bipartite intersection graph between horizontal and vertical streets: $\mathcal{G} = (H \cup V, E)$ where $v, h \in E$ if the segments v and h intersect. Clearly, the set E of edges corresponds to the set of junctions $\mathcal{J}$ (there are no other intersection points between streets because every two streets of P in the same direction are disjoint). In Figure 4.2a, the thin polyomino is decomposed into its horizontal and vertical streets. The corresponding bipartite graph is shown in Figure 4.2b, where horizontal streets (H) and vertical streets (V) are represented as the two vertex classes. We add an edge between a vertex in H and a vertex in V whenever the corresponding streets intersect in the polyomino. If a gate is located at the junction of a horizontal and a vertical street, the corresponding edge in the bipartite graph is marked in red. If a gate is not located at a junction, a dummy street is introduced so as to create an artificial junction, yielding a corresponding red edge in the bipartite graph.



(a) (b)

Figure 4.2: (a) Streets numbering. (b) Correspond bipartite graph.

4.1.1 NP-Hardness

The bipartite graph $\mathcal{G} = (H \cup V, E)$ belongs to a well-studied family of graphs known as *grid intersection graphs*. These graphs arise as intersection graphs of axis-aligned segments in the plane, where each vertex is represented by either a horizontal or a vertical segment, and two vertices are adjacent if and only if the corresponding segments intersect. A defining property of grid intersection graphs is that segments with the same orientation are required to be pairwise disjoint. As a consequence of this restriction, every grid intersection graph is bipartite, with one partition corresponding to horizontal segments and the other to vertical segments.

A more restrictive geometric representation is obtained when intersections are disallowed. A representation in which no two segments cross, and where adjacency is realized solely by segments touching each other at a point or along a subsegment, is called a *grid contact representation*. Graphs that admit such a representation are referred to as *grid contact graphs*. Since crossings are forbidden in this model, every grid contact graph is necessarily planar.

Interestingly, the class of grid contact graphs exactly captures planarity in the bipartite setting. De Fraysseix et al. [18] proved that every planar bipartite graph admits a grid contact representation. Together with the fact that every grid contact graph is planar, this establishes an equivalence between planar bipartite graphs and grid contact graphs. This connection provides a powerful bridge between geometric representations on the integer grid and classical structural properties of graphs.

Lemma 1. *The gate-cover problem in thin polyominoes with $\mathcal{X} = \mathcal{J}$ is polynomially equivalent to edge domination in grid intersection graphs.*

Proof. The set of gates equals to the set of junctions, which correspond to the intersections of the streets, i.e., to the edges of $\mathcal{G}$. The requirement that every gate is observed by a camera translates to the requirement that every junction belongs to a street that contains a camera. Mark the edges in $\mathcal{G}$ corresponding to camera-occupied junctions as *guarding edges*. The above correspondence yields that every edge of $\mathcal{G}$ must share a vertex with a guarding edge, i.e., the guarding edges form an *edge dominating set*.

The only difference is in measuring the input size. For G_P it is m – the number of unit squares in P, and for $\mathcal{G}$ it is the number of streets and junctions in P. Clearly, the

size of $\mathcal{G}$ is at most m. Vice versa, every grid intersection graph with n vertices has a grid representation with integer coordinates in the interval $[1..n]$, and hence the corresponding polyomino will have at most n^2 unit squares. □

The *edge domination problem* asks for a subset of edges in a graph with the property that every edge of the graph is either contained in the subset or shares at least one endpoint with an edge in the subset. In other words, the selected edges collectively "dominate" all edges of the graph via adjacency at vertices. The objective is to find such a dominating set of edges of minimum cardinality.

By Lemma 1, the gate-cover problem in thin polyominoes with $\mathcal{X} = \mathcal{J}$ is polynomially equivalent to the edge domination problem in grid intersection graphs. Consequently, to establish NP-hardness of the gate-cover problem, it suffices to show that edge domination is NP-hard for this class of graphs.

The prove will give a stronger statement. The reduction shows that edge domination remains NP-hard even when restricted to grid contact graphs. Through the equivalence established above, this implies NP-hardness of the gate-cover problem for thin polyominoes in which every junction is a T-shaped junction and all gates are located at junctions, that is, $\mathcal{X} = \mathcal{J}$. Yannakakis and Gavril [46] showed that edge domination is NP-hard for bipartite graphs and for planar graphs. Proof of Lemma 2 demonstrate that their reduction actually shows NP-hardness for planar bipartite graphs.

Lemma 2. *Edge domination is NP-hard for planar bipartite graphs of maximum degree 3.*

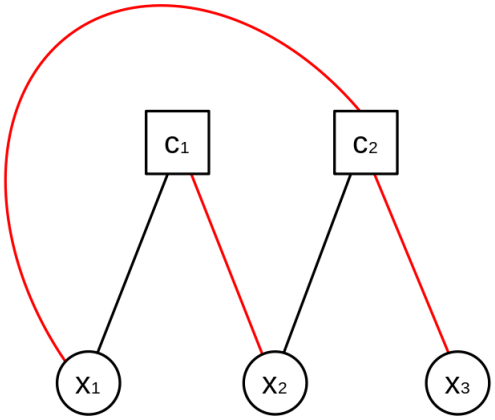
Proof. The reduction of Yannakakis and Gavril [46] can be applied with the modification that the reduction is not from 3-Satisfiability, but from a restricted variant of Planar-3-Satisfiability.

For a Boolean formula $\Phi = (X, C)$ in conjunctive normal form, where X is the set of variables and C is the set of clauses, the *incidence graph* G_Φ is defined as a bipartite graph with vertex set $X \cup C$, in which one part consists of the variable vertices X and the other of the clause vertices C. An edge connects a variable $x \in X$ to a clause $c \in C$ if and only if x appears in c, either as a positive literal x or as a negated literal $\neg x$. Formally,

$$G_\Phi = (X \cup C, \{xc : x \in X,\ c \in C,\ x \in c \text{ or } \neg x \in c\}).$$

The formula is called *planar* if its incidence graph is planar (An example is shown in Figure 4.3).

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$$



$$c_1 = x_1 \vee \neg x_2$$
$$c_2 = \neg x_1 \vee x_2 \vee \neg x_3$$

Figure 4.3: A planar bipartite graph and its corresponding CNF formula.

Satisfiability of planar formulas was proved NP-hard by Lichtenstein [33], in his reduction variables may occur more than three times, which is too much for this reduction. However, it may be assumed that the planar formula $\Phi = (X, C)$ used in the reduction is such that every clause contains two or three literals, and every variable occurs in exactly three clauses, once negatively and twice positively. NP-hardness of this variant of Planar-3-Satisfiability is proved in Fellows et al. [21].

Suppose given a formula Φ as described. For a variable $x \in X$, let us denote by a_x and b_x the two clauses in which x appears positively, and by c_x the clause in which x appears in negation. The graph $G(\Phi)$ is constructed as follows. For every clause $c \in C$, a clause gadget is added to $G(\Phi)$, consisting of two vertices v^c_+ and v^c_- connected by an edge (Figure 4.4).

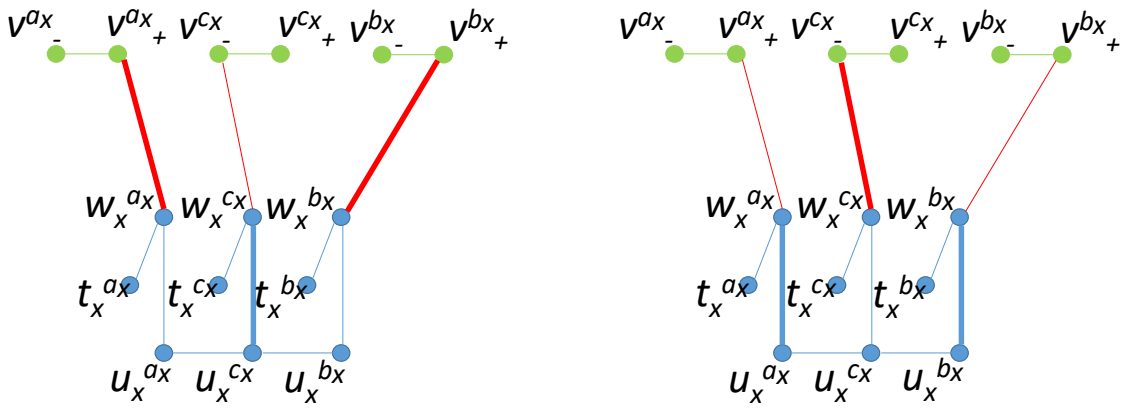


Figure 4.4: The variable gadget (marked in blue) and its connection to clause gadgets (marked in green) of clauses that contain this variable. The connecting edges are red. Matching edges that dominate all edges of the variable gadget are displayed in bold. The variant in the left corresponds to $\phi(x) = true$, the variant to the right corresponds to $\phi(x) = false$.

For every variable $x \in X$ that appears in a clause $c \in C$, A path is included - $u_x^c w_x^c t_x^c$ and form the variable gadget by adding the edges $u_x^{a_x} u_x^{c_x}$ and $u_x^{b_x} u_x^{c_x}$. The variable and clause gadgets are connected by edges $w_x^{a_x} v_+^{a_x}$, $w_x^{b_x} v_+^{b_x}$ and $w_x^{c_x} v_-^{c_x}$. The graph constructed in this way is bipartite, with classes of bipartition $\bigcup_{x \in X} \{u_x^{a_x}, t_x^{a_x}, u_x^{b_x}, t_x^{b_x}, w_x^{c_x}\} \cup \bigcup_{c \in C} \{v_+^c\}$ and $\bigcup_{x \in X} \{u_x^{c_x}, t_x^{c_x}, w_x^{a_x}, w_x^{b_x}\} \cup \bigcup_{c \in C} \{v_-^c\}$. Since the variable and clause gadgets are acyclic, a non-crossing drawing of $G(\Phi)$ can be easily constructed from a planar drawing of G_Φ. Thus $G(\Phi)$ is planar and bipartite (Figure 4.5).

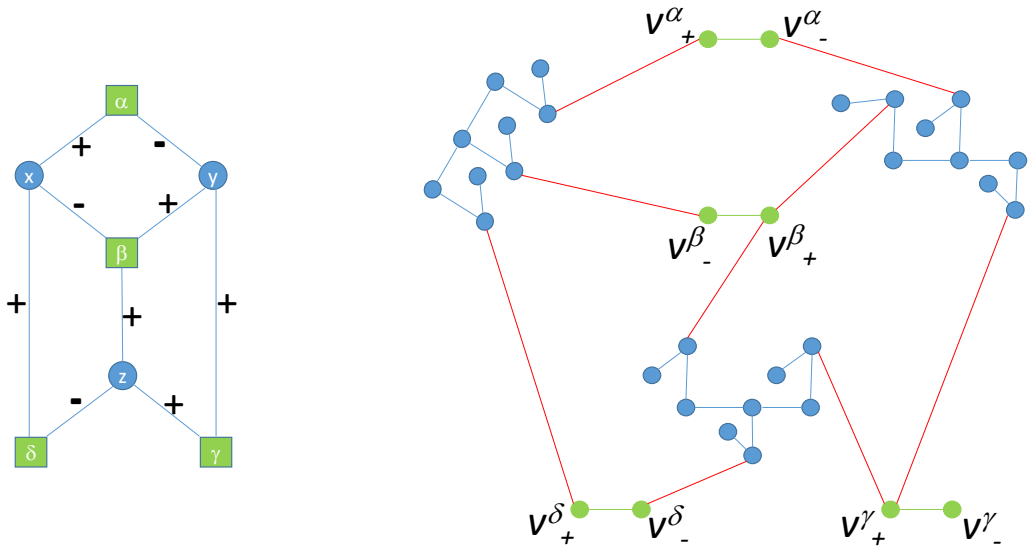


Figure 4.5: The construction of the graph $G(\Phi)$ for a formula $\Phi = \alpha \wedge \beta \wedge \gamma \wedge \delta$ with $\alpha = (x \vee \neg y)$, $\beta = (\neg x \vee y \vee z)$, $\gamma = (y \vee z)$ and $\delta = (x \vee \neg z)$. A planar non-crossing drawing of $G(\Phi)$ in the right is derived from a non-crossing drawing of the incidence graph G_Φ of Φ depicted in the left.

It is claimed that $G(\Phi)$ has an edge dominating set of size $3|X|$ if and only if Φ is satisfiable. Let us repeat the argument of Yannakakis [46]. If Φ is satisfied by an assignment $\phi : X \to \{true, false\}$, then

$$\bigcup_{x \in X, \phi(x)=true} \{w_x^{a_x} v_+^{a_x}, w_x^{b_x} v_+^{b_x}, u_x^{c_x} w_x^{c_x}\} \cup \bigcup_{x \in X, \phi(x)=false} \{w_x^{c_x} v_-^{c_x}, u_x^{a_x} w_x^{a_x}, u_x^{b_x} w_x^{b_x}\}$$

is an edge dominating set of size $3|X|$. If, on the other hand, $G(\Phi)$ has an edge dominating set F of size at most $3|X|$, F cannot include any edge $v_+^c v_-^c$ from the clause gadgets, because the matching $\{w_x^c t_x^c : x \in c \in C\}$ of size $3|X|$ must be dominated by edges from the variable gadgets or the edges connecting the variable gadgets to the clause ones. Thus for every clause c, at least one of the connecting edges has to be in F, and the a truth valuation ϕ defined by $\phi(x) = true$ iff $w_x^c v_+^c \in F$ satisfies Φ.
this truth valuation is legal, since it can be shown that $|F| = 3|X|$ implies that for every variable $x \in X$, either

$$F \cap \{w_x^{a_x} v_+^{a_x}, w_x^{b_x} v_+^{b_x}, w_x^{c_x} v_-^{c_x}\} = \{w_x^{c_x} v_-^{c_x}\}$$

or

$$F \cap \{w_x^{a_x} v_+^{a_x}, w_x^{b_x} v_+^{b_x}, w_x^{c_x} v_-^{c_x}\} \subseteq \{w_x^{a_x} v_+^{a_x}, w_x^{b_x} v_+^{b_x}\}$$

with equality being admissible in the latter inclusion. □

This concludes the proof of the following theorem.

Theorem 1. *The gate-cover problem in thin polyominoes is NP-hard, even if no unit square of the polyomino is adjacent to more than three other ones, and the set of gates equals the set of junctions.*

4.2 Bounded vision

Consider cameras with *bounded vision*: a camera can observe (guard) only those gates that lie within a fixed range. More precisely, for a given integer $k \geq 1$, a camera can see only gates that are at hop-distance at most k from its location, where distance is measured along horizontal or vertical grid paths.

Let G be a *grid graph*, that is, an induced subgraph of the integer grid. The graph is given together with its natural planar embedding, in which every vertex is placed at an integer grid point, every edge has unit length, and two vertices are adjacent exactly when they are at unit Manhattan distance. Such graphs naturally arise from polyominoes by associating a vertex with each cell and edges between horizontally or vertically adjacent cells.

A subset of vertices $C \subseteq V(G)$ is called *k-range dominating* if every vertex $v \notin C$ can reach some vertex of C by a *straight* path of length at most k, where the path is either entirely horizontal or entirely vertical. Intuitively, a dominating vertex can "see" along grid lines up to distance k, but cannot turn corners.

This notion differs from the classical concept of *k-distance domination*, where each vertex must be within graph distance at most k from a dominating vertex, but the connecting path may change direction arbitrarily. In contrast, k-range domination imposes a stronger geometric constraint: visibility must follow a single row or column of the grid.

When considering a polyomino P, guarding all its cells is equivalent to guarding all its gates. Placing cameras in P corresponds to selecting vertices in the associated grid graph G_P. A camera placed at a cell guards exactly those cells that lie within a horizontal or vertical range of at most k cells. Consequently, a minimum set of cameras that guards all gates of P corresponds exactly to a minimum k-range dominating set of G_P, and vice versa.
This correspondence yields the following observation.

Observation 1. *The gate-cover problem with bounded range k in thin polyominoes is equivalent to the k-range domination problem in grid graphs of girth greater than* 4.

Recall that the *girth* of a graph is defined as the length of its shortest cycle. In the context of grid graphs derived from polyominoes, the smallest possible cycle is a 4-cycle, corresponding to a 2×2 block of cells.

Thin polyominoes exclude such 2×2 blocks by definition. As a consequence, the grid graphs associated with thin polyominoes contain no cycles of length 4. Therefore, every cycle in these graphs has length strictly greater than 4, and the associated grid graphs are exactly those of girth greater than 4.

4.2.1 NP-Hardness

It is shown that bounded-range gate-cover is NP-hard for any value of k.

Theorem 2. *The gate-cover problem with bounded range k is NP-hard on thin polyominoes for every $k \geq 1$.*

The case $k = 1$ is established via a reduction from the domination problem in cubic planar graphs (Lemma 3), while for $k > 1$ a reduction from a variant of 3-SAT is employed (Lemma 4).

Lemma 3. *The gate-cover problem with bounded range $k = 1$ is NP-hard on thin polyominoes.*

Proof. The reduction is from the Dominating Set problem in planar cubic graphs, whose NP-hardness was established in [26] and later attributed in [47], and also follows from [31].

Let G be a planar cubic graph. The construction starts with an orthogonal drawing of G, in which every vertex is placed at a grid point with integer coordinates and every edge is represented by a piecewise-linear curve consisting exclusively of horizontal and vertical segments. All bending points of edges are also placed at integer grid points. Such an orthogonal drawing always exists and can be constructed in linear time [28].

The grid is then refined so that the coordinates of all vertices and bending points become divisible by 8. This refinement preserves planarity and relative geometry while ensuring sufficient spacing for subsequent local modifications. Let G' denote the grid graph induced by this refined embedding of G. Since the refinement increases all dimensions by only a constant factor, both the area of the embedding and the size of G' remain polynomial in the size of G.

Next, certain paths in G' are locally modified. Consider a maximal path whose internal vertices all have degree 2 and whose endpoints have degree 3. Each such path corresponds to an edge of the original graph G. For an edge $e \in E(G)$, the corresponding path in G' is denoted by P_e. By construction, P_e consists of one or more horizontal or vertical segments, each of length at least 8.

The goal of the modification is to control the interaction between these paths and 1-range domination. Recall that in k-range domination, a vertex can dominate only vertices that lie on the same row or column at distance at most k. Ensuring that the length of each path P_e is congruent to 1 modulo 3 guarantees that exactly one well-positioned vertex on the path can dominate all relevant vertices along P_e under 1-range visibility, while preventing unintended domination from neighboring paths.

Since the initial length of each path P_e is a multiple of 8, its length modulo 3 can be only 0, 1, or 2. If the length is already congruent to 1 modulo 3, the path is left unchanged. If the length is congruent to 2 modulo 3, a single short detour is introduced along one straight segment of the path, for example near its beginning. If the length is divisible by 3, a double detour is introduced instead. These detours locally increase the length of the path while preserving its endpoints, planarity, and overall grid alignment.

The three possible cases are illustrated in Figure 4.6, where the top, middle, and bottom drawings correspond to paths whose lengths are congruent to 1, 2, and 0 modulo 3, respectively.

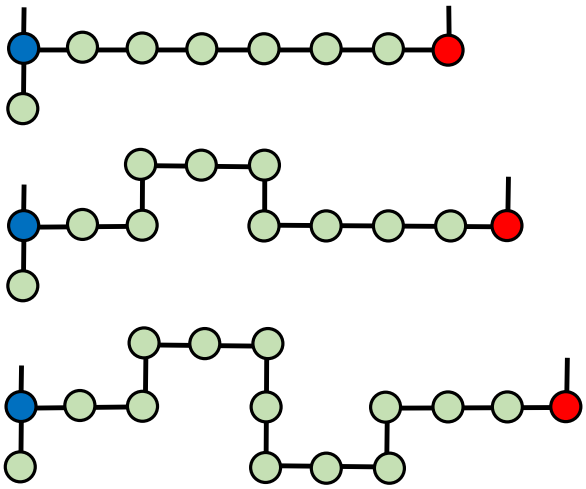


Figure 4.6: Local modifications of paths in G' used to enforce path lengths congruent to 1 modulo 3. Blue vertices represent vertices of degree 3, while red vertices correspond to vertices of G' inherited from bending points in the orthogonal drawing of G.

Let G'' denote the graph obtained from G' after all path modifications have been applied. For each edge $e \in E(G)$, recall that the corresponding path in G'' has length $1 + 3\ell_e$. The following claim relates the domination numbers of G and G''. Here, $\gamma(H)$ denotes the domination number of a graph H, that is, the minimum cardinality of a dominating set in H.

$$\gamma(G'') = \gamma(G) + \sum_{e \in E(G)} \ell_e.$$

This immediately implies that computing the domination number of G'' is NP-hard, since $\gamma(G)$ can be recovered directly from $\gamma(G'')$.

To prove the claim, consider a minimum dominating set C'' of G'' that contains a maximum possible number of vertices of degree 3. Define:

$$C = C'' \cap V(G),$$

C consists of those dominating vertices that correspond to original vertices of G. It is shown that C is a dominating set of G.

Indeed, let u be any vertex of G with $u \notin C$. Since C'' is a dominating set of G'', the vertex u must be dominated by some vertex lying on a path P_e corresponding to an edge $e = uv \in E(G)$. Consider this path P_e, oriented from u toward v.

Along P_e, the dominating vertices can be shifted in a systematic way - they are placed at every third vertex along the path, starting from the vertex closest to u. In other words, the first, fourth, seventh, and so on vertices of P_e belong to C'', ensuring that no inner vertex of the path is dominated more than once, see Figure 4.7. This arrangement uses exactly ℓ_e dominating vertices on P_e.

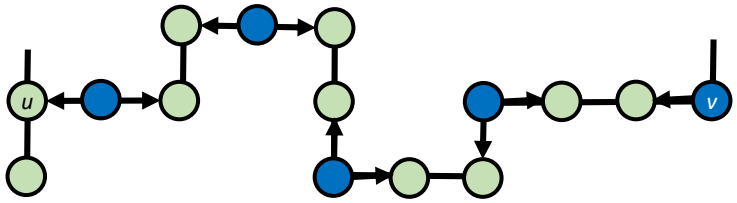


Figure 4.7: Propagation of dominating vertices along the path P_{uv}. Vertices in the dominating set are shown in blue, with arrows indicating the vertices they dominate.

Because the length of P_e is $1+3\ell_e$, the second-to-last inner vertex is dominated by the third-to-last one. The last inner vertex is either included in C'' or is dominated by the endpoint v. If $v \notin C''$, then replacing the last inner dominating vertex of P_e by v yields another dominating set of the same size but with more vertices from $V(G)$, contradicting the maximality of $|C'' \cap V(G)|$. Hence, $v \in C''$, and v dominates u in G.

Therefore, every vertex of G is dominated by C, and C is a dominating set of G. Since exactly ℓ_e vertices of C'' lie on each path P_e, it follows that

$$|C'' \cap V(G)| = |C''| - \sum_{e \in E(G)} \ell_e.$$

Consequently,

$$\gamma(G) \leq |C'' \cap V(G)| = \gamma(G'') - \sum_{e \in E(G)} \ell_e,$$

which proves the claim.

Conversely, any dominating set of G can be extended to a dominating set of G'' by adding exactly ℓ_e vertices on each path P_e, for all $e \in E(G)$. This yields a dominating set of size $\gamma(G) + \sum_{e \in E(G)} \ell_e$ in G'', establishing the reverse inequality. □

Lemma 4. *The gate-cover problem with bounded range k is NP-hard on thin polyominoes for every $k > 1$.*

Proof. The reduction is from Planar-(2-or-3)-SAT, a variant of the satisfiability problem in which the incidence graph of the input formula is planar, each variable occurs exactly twice negated and once positively, and every clause contains either two or three literals. A truth assignment satisfies the formula if every clause contains at least one positively valued literal. This satisfiability problem is NP-hard [21].

Given a Boolean formula Φ, the construction begins with an orthogonal drawing of its incidence graph, following the same approach as in the previous case. Variable vertices are replaced by *variable gadgets*, clause vertices by *clause gadgets*, and edges by *linking gadgets*. To enable a uniform geometric construction, the drawing is scaled so that all vertex coordinates and bending points are divisible by

$$t = 2k + 1,$$

and all components are placed sufficiently far apart to avoid unintended interactions.

Variable gadget. The variable gadget is shown in Figure 4.8. The central octagonal structure formed by the green, blue, and red vertices is referred to as the *core* of the gadget. Each gray vertex represents a path of length $k-2$.

Let C be a k-range dominating set. Within the core of each variable gadget, at least two vertices must belong to C in order to dominate all four green vertices. To minimize the size of C, exactly two such vertices are chosen. These must be either the two blue vertices or the two red vertices.

If the blue vertices are chosen, the two pink vertices are left undominated. if the red vertices are chosen, the light-blue vertex remains undominated. These undominated vertices are referred to as the *ports* of the variable gadget. The left and right ports correspond to occurrences of the variable as a negated literal, while the bottom port corresponds to its positive occurrence. Selecting the blue vertices in C therefore represents a truth assignment of `true`, leaving the negative ports undominated, whereas selecting the red vertices represents a truth assignment of `false`, leaving the positive port undominated.

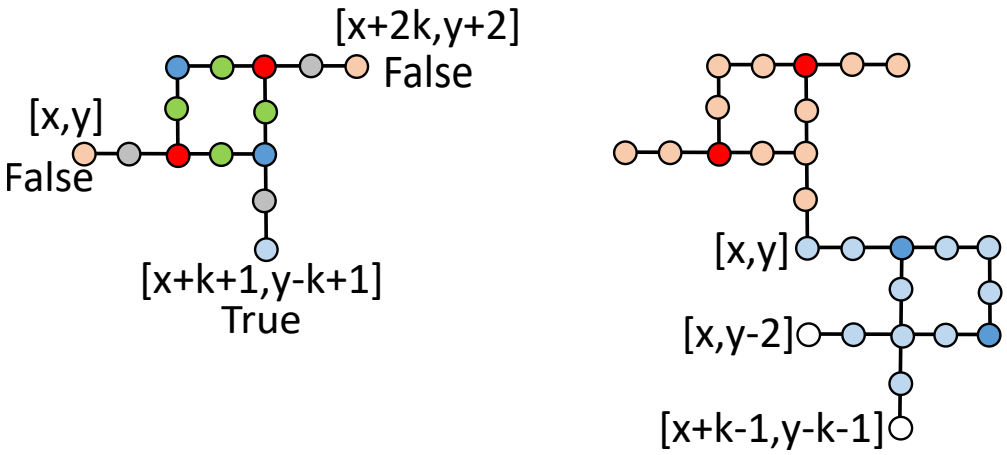


Figure 4.8: The variable gadget (left) and a reduced core used to adjust port coordinates (right).

Linking gadget. The linking gadget, illustrated in Figure 4.9, is a piecewise-linear path whose segments all have lengths divisible by t. Its role is to propagate the logical value `false`.

If one terminal vertex of the linking gadget is not dominated from outside and must therefore be dominated internally, the placement of dominating vertices along the gadget is forced in order to use the minimum possible number, namely the total length of the path divided by t. This forced placement leaves the opposite terminal vertex undominated, thereby propagating the `false` value, see Figure 4.9. If one terminal vertex has both coordinates divisible by t, then the same holds for every internal vertex of the linking gadget and, in particular, for the other terminal vertex.

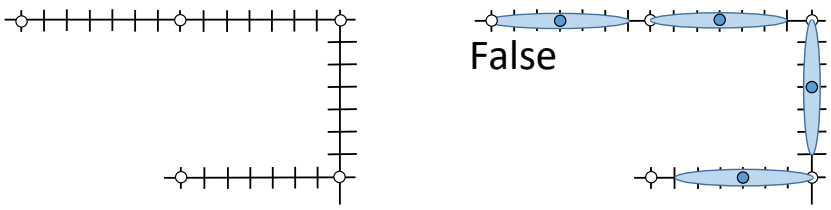


Figure 4.9: The linking gadget (left) and the propagation of a false value along it (right).

Clause gadget. Clause gadgets are simple paths. A clause with two literals is represented by a path of length t, while a clause with three literals is represented by a path of length $2t$. The terminal vertices of the path, and additionally the middle vertex in the case of a three-literal clause, are identified with the terminal vertices of the linking gadgets arriving from the corresponding variables. These vertices are called the *ports* of the clause gadget.

A two-literal clause gadget must contain at least one dominating vertex, and a three-literal clause gadget must contain at least two. These bounds are sufficient if and only if at least one port is dominated from outside the gadget, that is, from a linking gadget corresponding to a variable that is not evaluated to `false` in the clause. If one port has both coordinates divisible by t, then the remaining ports do as well.

Adjusting port coordinates. The remaining difficulty concerns the coordinates of the ports of the variable gadgets. Suppose the left port has coordinates (x, y). Then the right port has coordinates $(x{+}2k, y{+}2)$ and the bottom port has coordinates $(x{+}k{+}1, y{-}k{+}1)$. If both x and y are divisible by t, the right port has residue $(-1, 2)$ mod t and the bottom port has residue $\big((t+1)/2, (-t+3)/2\big)$ mod t.

Two tools are used to resolve this issue. The first is the insertion of a path of length t, corresponding to one segment of a linking gadget. This propagates the value `false` while preserving coordinate residues modulo t. It is primarily used to create space. The second tool is the reduced copy of the variable core shown in Figure 4.8 (right). This gadget propagates the `false` value from (x, y) to $(x, y{-}2)$ and diagonally to $(x{+}k{-}1, y{-}k{-}1)$.

Examining the variable gadget in Figure 4.8 (left), the pink vertices are seen to have coordinates of the same parity. If k is odd, the negative port has the same property. If k is even, diagonal propagation is applied, resulting in a new vertex at $(x + 2k, y - 2k)$, again preserving parity. Once all ports have even coordinates, repeated applications of the reduced core gadget (and its 90° rotation, if necessary) make all coordinates divisible by $2t$.

Putting everything together. Suppose the formula Φ contains v variables, c_2 clauses of size 2, and c_3 clauses of size 3. Assume that the construction uses ℓ linking gadgets (including those for variable–clause connections and for coordinate adjustments) and h reduced-core propagation gadgets. Then every k-range dominating set of the resulting graph G_Φ has size at least

$$2v + \ell + c_2 + 2c_3 + 2h.$$

This bound is tight if and only if the formula Φ is satisfiable. Finally, the girth of G_Φ is greater than 4, and hence G_Φ corresponds to a thin polyomino.

□

5 Algorithmic results

5.1 A 2-approximation algorithm

Since NP-hard problems are unlikely to admit polynomial-time exact algorithms, approximation algorithms provide a natural and practically relevant alternative. They aim to compute solutions efficiently while guaranteeing provable bounds on their quality relative to an optimal solution. Studying approximation algorithms for NP-hard problems therefore allows one to balance computational tractability with solution quality, and often yields deeper structural insights into the problem itself.

In the gate-cover problem, the objective is to cover all gates by cameras, that is, to find a minimum set C of unit squares (camera locations) in a polyomino P such that every gate in $\mathcal{X}$ lies on the same street as at least one camera from C. As an intermediate step, consider the related problem of covering gates by streets, find a minimum set S of streets in P such that every gate in $\mathcal{X}$ belongs to at least one street in S.

An optimal solution to the street-cover problem yields a factor-2 approximation for the camera-cover problem. Let OPT_S denote an optimal set of streets covering all gates, and let OPT_C denote an optimal set of cameras. Clearly,

$$|OPT_C| \leq |OPT_S|,$$

since placing a single camera on each street in OPT_S suffices to see all gates on that street. Conversely,

$$|OPT_S| \leq 2|OPT_C|,$$

because each camera in OPT_C lies at the intersection of at most one horizontal and one vertical street, and replacing that camera by these two streets covers exactly the same set of gates. Thus, solving the street-cover problem optimally provides a 2-approximation for the original problem.

When $\mathcal{X} = \mathcal{J}$, that is, when all gates lie at street junctions, the problem of finding a minimum set of streets covering all gates is equivalent to computing a *minimum vertex cover* in the bipartite graph $\mathcal{G}$. In this graph, vertices correspond to horizontal and vertical streets, and each gate induces an edge between the two streets intersecting at that gate.

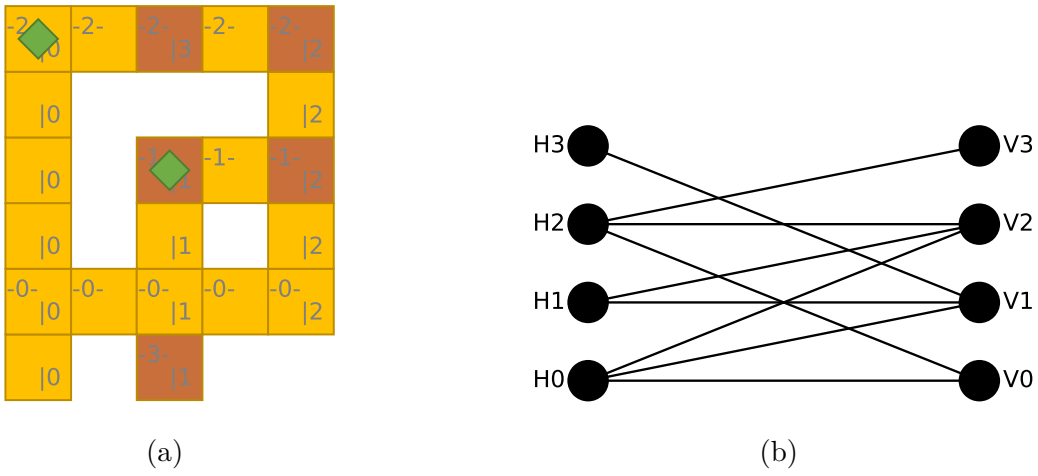
When $\mathcal{X} \neq \mathcal{J}$, gates may lie on arbitrary unit squares of P, and the construction of $\mathcal{G}$ must be extended accordingly. The resulting graph is denoted by

$$\mathcal{G}^{\mathcal{X}} = \big((H \cup \hat{H}) \cup (V \cup \hat{V}), E^{\mathcal{X}}\big),$$

and is defined as follows (Figure 5.1b).
The vertex sets H and V consist of all horizontal and vertical streets of P, as before. For every gate that lies at the intersection of a horizontal street $h \in H$ and a vertical street

$v \in V$, the edge (h, v) is added to $E^{\mathcal{X}}$. Edges corresponding to junctions that are not gates are not included.

To represent gates that do not lie at junctions, additional vertices are introduced. For each gate $x \in \mathcal{X} \setminus \mathcal{J}$, exactly one *dummy street* is added. If x lies on a horizontal street $S_h(x)$, a new vertex v_x is added to $\hat{V}$ together with the edge $(S_h(x), v_x)$. Otherwise, x lies on a vertical street $S_v(x)$, and a new vertex h_x is added to $\hat{H}$ together with the edge $(h_x, S_v(x))$. In this way, every gate in $\mathcal{X}$ corresponds to exactly one edge in $\mathcal{G}^{\mathcal{X}}$.



(a) (b)

Figure 5.1: (a) A polyomino with 3 horizontal and 3 vertical streets. An optimal solution has size two, for example, using the cameras marked in green diamonds. (b) The graph $\mathcal{G}^{\mathcal{X}}$. H_3 and V_3 are dummy streets. Optimal vertex cover has size 4.

Some vertices of $\mathcal{G}^{\mathcal{X}}$ correspond to dummy streets rather than actual streets of P. By construction, every dummy vertex has degree 1. Consequently, a minimum vertex cover of $\mathcal{G}^{\mathcal{X}}$ never contains both a dummy vertex and its unique neighbor. If a minimum vertex cover contains a dummy vertex, it can be replaced by its neighbor without increasing the size of the cover. It follows that minimum vertex covers of $\mathcal{G}^{\mathcal{X}}$ correspond exactly to minimum sets of actual streets in P that cover all gates.

Lemma 5. *Let S be a set of streets in P. Then S covers all gates in $\mathcal{X}$ if and only if the corresponding set of vertices forms a vertex cover of $\mathcal{G}^{\mathcal{X}}$.*

The graph $\mathcal{G}^{\mathcal{X}}$ is bipartite and has $|\mathcal{X}|$ edges. A minimum vertex cover in such a graph can be computed in time $O(|\mathcal{X}|^{1+o(1)})$ [11]. This yields the following result.

Theorem 3. *Given a thin polyomino P with m unit squares and a set $\mathcal{X}$ of gates in P, a set C of cameras covering all gates in $\mathcal{X}$ can be computed in time $O\big(m + |\mathcal{X}|^{1+o(1)}\big)$, such that $|C| \leq 2|OPT_C|$, where OPT_C denotes an optimal solution to the gate-cover problem.*

5.2 Thin polyominoes with bounded treewidth

Treewidth is a structural parameter that measures how close a graph is to being a tree. A tree has treewidth 1, and larger values indicate increasingly complex interconnections. Formally, a *tree decomposition* of a graph $G = (V, E)$ is a tree T whose nodes are associated with subsets of V, called *bags*, such that: (i) for every vertex $v \in V$, the bags

containing v form a connected subtree of T, and (ii) for every edge $(v, w) \in E$, there exists a bag containing both v and w. The width of a tree decomposition is the size of its largest bag minus one, and the treewidth $tw(G)$ is the minimum width over all possible tree decompositions of G.

Let P be a thin polyomino and let $\mathcal{X}$ denote a set of gates. Consider the dual graph G_P of P, and assume that G_P has bounded treewidth t. To simplify the structure without affecting the parameter, the graph is first reduced by contracting every edge (u, w) such that at least one of u, w does not belong to $\mathcal{J} \cup \mathcal{X}$ (see Figure 5.2). The resulting graph, still denoted G_P, is a minor of the original dual graph and therefore has treewidth at most t. After this reduction, the vertex set of G_P consists precisely of $\mathcal{J} \cup \mathcal{X}$.

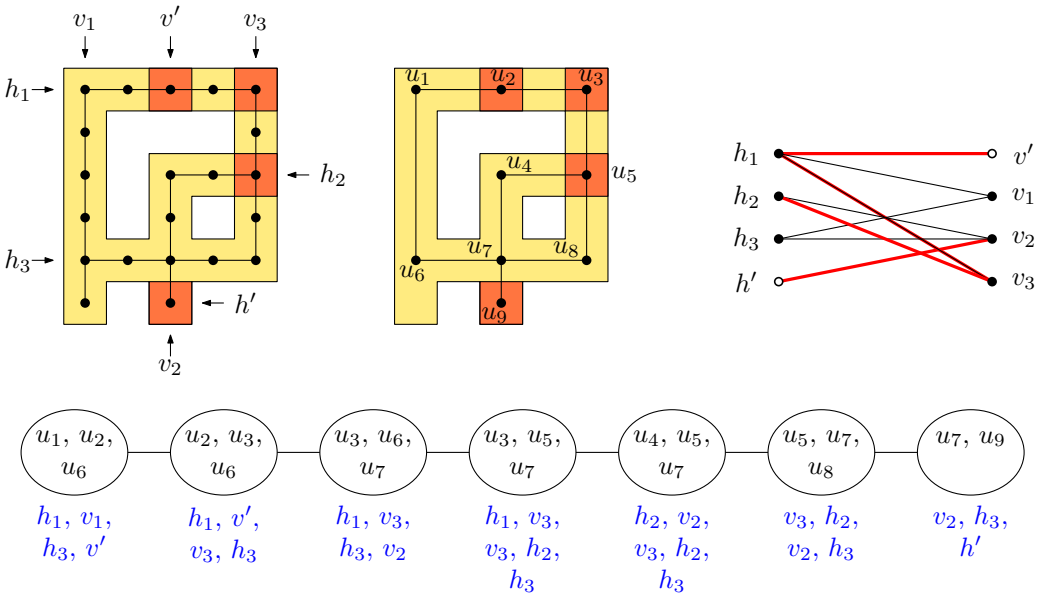


Figure 5.2: Reduction of the dual graph while preserving bounded treewidth.

Lemma 6. *Let P be a thin polyomino whose dual graph has treewidth t. Then the treewidth of the associated bipartite graph $\mathcal{G}^{\mathcal{X}\cup\mathcal{J}}$ is at most $2t$.*

Proof. Let T be a tree decomposition of width t for the cleaned dual graph G_P. A tree decomposition T' for the bipartite graph $\mathcal{G}^{\mathcal{X}\cup\mathcal{J}}$ can be constructed From T as follows. Each vertex u of G_P appearing in a bag of T is replaced by at most two street vertices, $S_h(u)$ and $S_v(u)$ (one of which may be a dummy vertex in $\mathcal{G}^{\mathcal{X}\cup\mathcal{J}}$). Since each original bag contains at most $t+1$ vertices, each new bag contains at most $2(t+1)$ vertices, and therefore the width of T' is at most $2t$ (see: Figure 5.2).

It remains to verify the decomposition properties. Every edge of $\mathcal{G}^{\mathcal{X}\cup\mathcal{J}}$ corresponds to a vertex of G_P, and hence is contained in a bag of T' whenever the corresponding vertex appears in T. For the connectivity condition, consider a street vertex h of $\mathcal{G}^{\mathcal{X}\cup\mathcal{J}}$. Let $u_1, \ldots, u_k$ be the vertices of G_P that define this street. These vertices form a path in G_P, and therefore the bags of T containing them form a connected subtree. By construction, h appears in precisely the corresponding bags of T', and thus the connectivity condition is preserved. □

This result establishes that bounded treewidth of the dual graph is preserved, up to a constant factor, in the guarding formulation. Since the corresponding edge-domination

formulation is MSO_2-definable, bounded treewidth yields fixed-parameter tractability by the standard bounded-treewidth meta-theorems [14]. Consequently, guarding thin polyominoes with bounded treewidth can be approached using standard dynamic programming techniques over tree decompositions. The lemma therefore provides the structural foundation for fixed-parameter algorithms in this setting. We therefore obtain the following theorem.

Theorem 4. *Let P be a thin polyomino whose dual graph has treewidth t. Then for any set of gates $\mathcal{X}$, we can solve the gate-cover-problem in time $O(f(t)n)$ for some computable function f.*

5.3 A separator based exact algorithm

In this section we present another exact algorithm for the gate-cover-problem which is based on the use of *orthogonal line separators*. Although the problem admits an exact algorithm on instances of bounded treewidth, the following algorithm remains important because its running time is explicitly guaranteed. By contrast, the treewidth-based algorithm has a running time of the form $O(f(t)n)$, where the function f may be large and may lead to worse performance when the treewidth is not small. Separator techniques have recently been employed to design exponential-time exact algorithms for a variety of geometric and graph-theoretic problems [17].

A result of Carmi et al. [9] guarantees the existence of a horizontal or vertical grid line that intersects $O(\sqrt{n})$ unit squares of a polyomino P, and such that each side of the line contains at most $\frac{4}{5}n$ unit squares of P. Such a line is referred to as a *separator*.

Consider a vertical separator line ℓ. The problem can be decomposed into at most $3^{O(\sqrt{n})}$ subproblems, each defined on a polyomino of size at most $\frac{4}{5}n$. The branching arises from the treatment of unit-square edges lying on the separator. For each such edge, there are three possible configurations (Figure 5.3):

1. a camera must be placed **above** the edge, in the same vertical street.
2. a camera must be placed **below** the edge, in the same vertical street.
3. there is no guarantee that a camera is place in this vertical street.

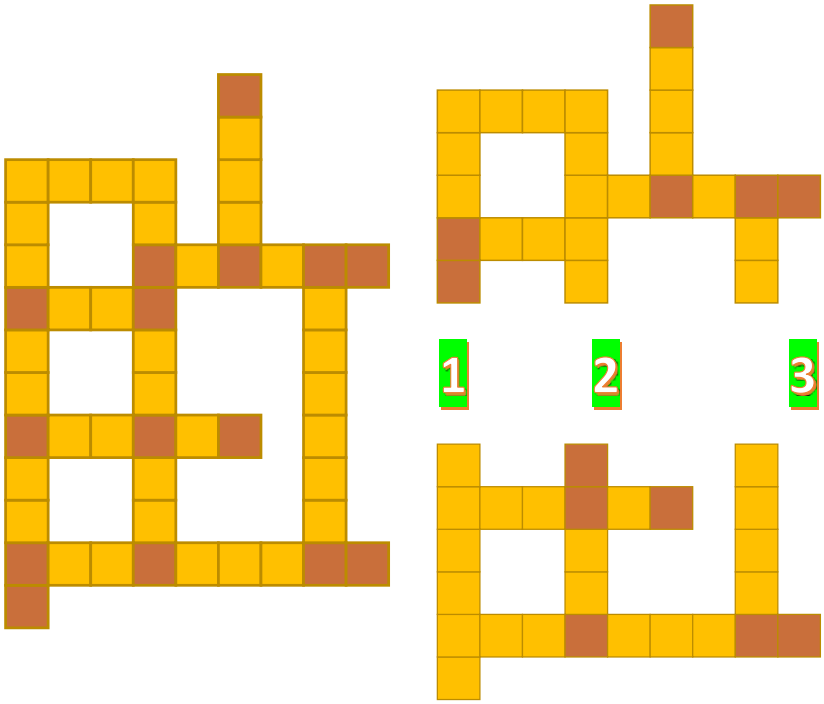


Figure 5.3: The three possible separator cases.

For subproblems of the third type, the polyomino remains unchanged. In subproblems of the first type, a single unit square with a gate is added immediately above the separator, and all gates in the same column below the separator are removed. Symmetrically, for subproblems of the second type, a single unit square with a gate is added below the separator, and all gates in the same column above the separator are removed. These modifications ensure that camera placements are consistent with the chosen configuration along the separator while preserving the structure of the problem.

The recursive decomposition continues until the subproblems reach a constant size, at which point the optimal solution can be computed by exhaustive enumeration. Since each recursive call produces subinstances that are strictly smaller, the recursion terminates after $O(\log n)$ levels.

For each branch induced by a separator, the modifications applied along the separator encode a separator configuration, specifying the relevant camera placement decisions on the boundary. This configuration is local and affects only the interface between the two sides of the separator. Consequently, after the subproblems are solved independently, their solutions can be combined without conflict. The camera placements on each side are consistent with the separator configuration, and together they form a feasible solution for the original instance corresponding to that branch.

The value of an optimal solution for a given branch is obtained by summing the optimal solutions of its subproblems, plus the number of cameras whose placement was fixed along the separator. Among all branches generated by the separator, the branch with minimum total cost is selected. This guarantees optimality, since every possible configuration of camera placements intersecting the separator is explicitly considered.

By exploring all separator branches and recursively solving the resulting subproblems, the algorithm enumerates all relevant global configurations in a structured way. The minimum solution returned by the recursion therefore corresponds to a minimum-size set of cameras guarding the entire polyomino, yielding an exact solution to the problem.

Running time

The running time follows from a standard separator recurrence. Let $T(n)$ denote the maximum time needed to solve an instance on a polyomino with n unit squares. By [9], there exists a horizontal or vertical grid line separator ℓ that intersects $m = O(\sqrt{n})$ unit squares of P, and such that each side of ℓ contains at most $\frac{4}{5}n$ unit squares. Along ℓ, at most m unit-square edges of P lie on the separator, and each such edge induces three branching options (camera above, camera below, or no commitment). Hence, the number of branches is at most $3^m = 3^{O(\sqrt{n})}$.

For each branch, the separator splits the instance into at most two independent subinstances, one on each side of ℓ, whose sizes are at most $\frac{4}{5}n$. The local modifications near ℓ (adding one square with a gate and removing gates in the corresponding column segment) take polynomial time and do not increase the instance size by more than a constant per separator edge. Therefore,

$$T(n) \;\le\; 3^{O(\sqrt{n})}\big(T(n_1) + T(n_2)\big) \;+\; n^{O(1)}, \qquad \text{where } n_1, n_2 \le \tfrac{4}{5}n \text{ and } n_1 + n_2 \le n.$$

In particular, using the coarse bound $T(n_1) + T(n_2) \le 2\,T(\frac{4}{5}n)$ yields

$$T(n) \;\le\; 2 \cdot 3^{c\sqrt{n}} \cdot T\big(\tfrac{4}{5}n\big) \;+\; n^{O(1)} \tag{5.1}$$

for some constant $c > 0$.

Unrolling (5.1) for $L = O(\log n)$ levels, the instance size decreases geometrically: $n_i \le (\frac{4}{5})^i n$. At level i, the branching factor contributes at most $3^{c\sqrt{n_i}} \le 3^{c\sqrt{n}\,(\sqrt{4/5})^{\,i}}$. Thus the product of branching factors over all levels is bounded by

$$\prod_{i=0}^{L-1} 3^{c\sqrt{n}\,(\sqrt{4/5})^{\,i}} \;=\; 3^{c\sqrt{n}\sum_{i=0}^{L-1}(\sqrt{4/5})^{\,i}} \;\le\; 3^{c\sqrt{n}\cdot\frac{1}{1-\sqrt{4/5}}} \;=\; 3^{O(\sqrt{n})}.$$

The additional multiplicative factor $2^L = n^{O(1)}$ coming from the leading 2 in (5.1), together with the polynomial work per node, is absorbed in the $O^*(\cdot)$ notation. Consequently,

$$T(n) = O^*\Big(3^{O(\sqrt{n})}\Big),$$

and, in particular, the algorithm runs in subexponential time $O^*(3^{\Theta(\sqrt{n})})$.

That conclude the following theorem.

Theorem 5. *There exists an exact algorithm for the gate-cover-problem that runs in time $O^*(3^{\Theta(\sqrt{n})})$ (where $*$ in $O^*(\cdot)$ hides some polynomial factors).*

5.4 A PTAS for the k-bounded version

For bounded vision and constant k, a PTAS can be obtained via local search. Local search is a heuristic method for solving NP-hard optimization problems. A local-search-based framework for obtaining PTASs for NP-hard geometric optimization problems was introduced, independently, by Chan and Har-Peled [10] and Mustafa and Ray [36]. Aschner et al. [3] generalize this framework by extending its analysis to additional families of graphs, beyond the family of planar graphs, and one of their applications is guarding a polygon under some shallowness assumption. More precisely, their approach implies the

following. Let P be a polygon, G a set of potential guards (points) in P, and X a set of points in P that needs to be guarded. If every point in X is only seen by a constant number of points from G, then the local search framework yields a PTAS for finding the minimum number of points from G to guard X. In this setting, every gate can be seen by $O(k)$ cameras. This yields the following theorem.

Theorem 6. *There exists a PTAS for the gate-cover problem in thin polyominoes with range k, for any constant k.*

6 Greedy heuristics and experimental results

This chapter complements the theoretical results of the thesis by studying practical algorithmic approaches for the Gate-Cover Problem. We present greedy and heuristic algorithms for both the unbounded and bounded-visibility models, and evaluate their performance against optimal solutions computed by 0-1 linear programming. The experiments demonstrate that, despite unfavorable worst-case behavior, carefully designed heuristics produce near-optimal solutions across synthetic city instances and a real urban case study.

6.1 Lower bounds for the basic greedy heuristic

Consider the following greedy approach for the gate-cover problem. Let P be a polyomino consisting of m unit squares, let $\mathcal{X}$ be a set of gates, and let c be a camera (unit square) placed in P. Denote by $\mathrm{Vis}(c) \subseteq \mathcal{X}$ the set of gates seen by camera c.

The algorithm maintains a set S of selected cameras, initially empty. As long as $\mathcal{X}$ is nonempty, a camera c maximizing $|\mathrm{Vis}(c)|$ is selected (if several cameras achieve the maximum, one of them is chosen arbitrarily). The camera c is added to S, and all gates in $\mathrm{Vis}(c)$ are removed from $\mathcal{X}$. The process terminates once all gates are removed, at which point S is a gate-cover for $(P, \mathcal{X})$.

A lower bound of $\frac{3}{2}$OPT

In this section we show a lower bound of $\frac{3}{2}$OPT on the approximation ratio of the above greedy approach. Figure 6.1 illustrates a polyomino in which the greedy algorithm selects three cameras, while an optimal solution uses only two. In this instance, the first greedy choice is unique, and no tie-breaking ambiguity arises. The resulting approximation ratio is therefore exactly 3/2. This example shows that the greedy strategy cannot guarantee an approximation factor better than 3/2 in general.

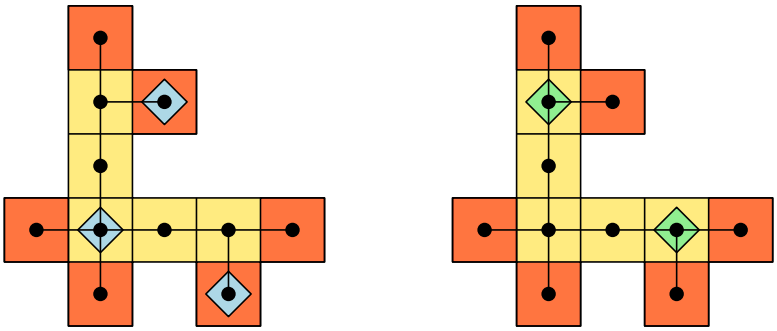


Figure 6.1: Left: a solution produced by the greedy algorithm. Right: an optimal solution.

A lower bound $(2-\varepsilon)OPT$

A more subtle behavior appears when tie-breaking is involved. Figure 6.2 presents a family of instances in which arbitrary tie-breaking may lead the greedy algorithm arbitrarily close to a factor-2 approximation.

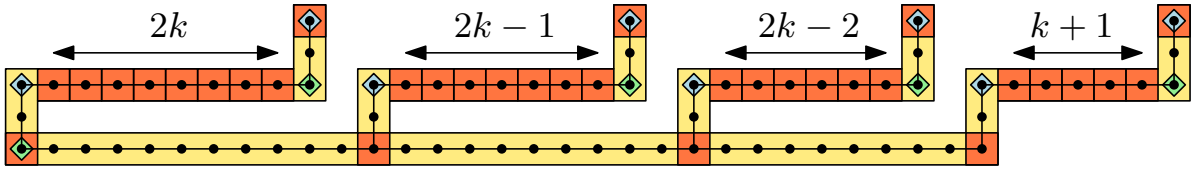


Figure 6.2: A counterexample family for the greedy algorithm, parameterized by $k = 4$.

The construction consists of a long horizontal street at the bottom, with k identical components attached to it. Each component contains two junctions, one colored blue and one green in the figure. Both junctions see exactly the same number of gates. Consequently, at each iteration of the greedy algorithm, there is a tie between selecting the blue or the green junction in the current component.

In the first iteration, both candidates see $2k+1$ uncovered gates. After one component is processed, the same situation arises in the next component, where the best candidates see $2k+2$ uncovered gates, and so on. If the greedy algorithm consistently resolves ties by selecting the blue junction, then after processing all components, the remaining blue junctions must also be selected, since their gates remain uncovered. This results in a total of $2k$ cameras.

In contrast, selecting the green junction in each component, together with a single green junction on the bottom street, yields a solution of size $k+1$, which is optimal. Let ALG denote the maximum, over all instances under consideration, of the number of cameras selected by the greedy algorithm, and let OPT denote the minimum number of cameras in an optimal solution. The ratio ALG/OPT therefore represents the worst-case approximation ratio

$$\frac{\text{ALG}}{\text{OPT}} = \frac{2k}{k+1} = 2 - \frac{2}{k+1}.$$

As k tends to infinity, the ratio converges to 2, and the resulting approximation ratio is

$$\text{ALG} = (2-\varepsilon)OPT$$

This family of examples demonstrates that, under arbitrary tie-breaking, the greedy algorithm may perform arbitrarily close to a factor-2 approximation. This motivates the use of carefully designed tie-breaking heuristics. The experimental evaluation therefore focuses on assessing the effectiveness of such heuristics across multiple classes of instances, including structured city layouts and randomly generated polyominoes, and on quantifying the resulting improvement in approximation quality relative to the baseline greedy strategy.

6.2 Greedy heuristics for unbounded vision

In this section, we presents different greedy algorithms operate on a bipartite graph derived from the gate-cover problem (see Figure 4.2). The construction abstracts the geometric structure of the polyomino into a combinatorial representation that preserves all relevant visibility relations.

Recall that $\mathcal{G} = (H \cup V, E)$ is the bipartite intersection graph of the streets of P, where H (resp. V) is the set of horizontal (resp. vertical) streets, and $\{v, h\} \in E$ if and only if the segments v and h intersect. As noted in Section 4.1, the edge set E corresponds exactly to the set of junctions $\mathcal{J}$. In Figure 4.2a, the thin polyomino is decomposed into its horizontal and vertical streets, and the corresponding bipartite graph is shown in Figure 4.2b. Edges corresponding to junctions that contain at least one gate are marked in red, indicating that the junction must be covered by the selected solution. If a gate is not located at a junction, a dummy street is introduced so as to create an artificial junction, yielding a corresponding red edge in the bipartite graph.

When a gate is located on a street but does not coincide with a junction, the representation is extended by introducing a perpendicular dummy street. This dummy street intersects the original street exactly at the location of the gate, thereby transforming the gate into an artificial junction. This transformation preserves the visibility structure of the instance and allows all gates to be treated uniformly as intersection points in the bipartite graph.

Two observations follow from this construction. First, even when gates are allowed to lie on streets and not exclusively at junctions, cameras in an optimal solution can always be assumed to be placed at junctions. Second, if a street contains multiple gates that do not lie at junctions, these gates can be represented by a single gate without affecting the size of an optimal solution. Accordingly, throughout the experimental evaluation, each street is assumed to contain at most one non-junction gate, and the bipartite representation is constructed under this assumption.

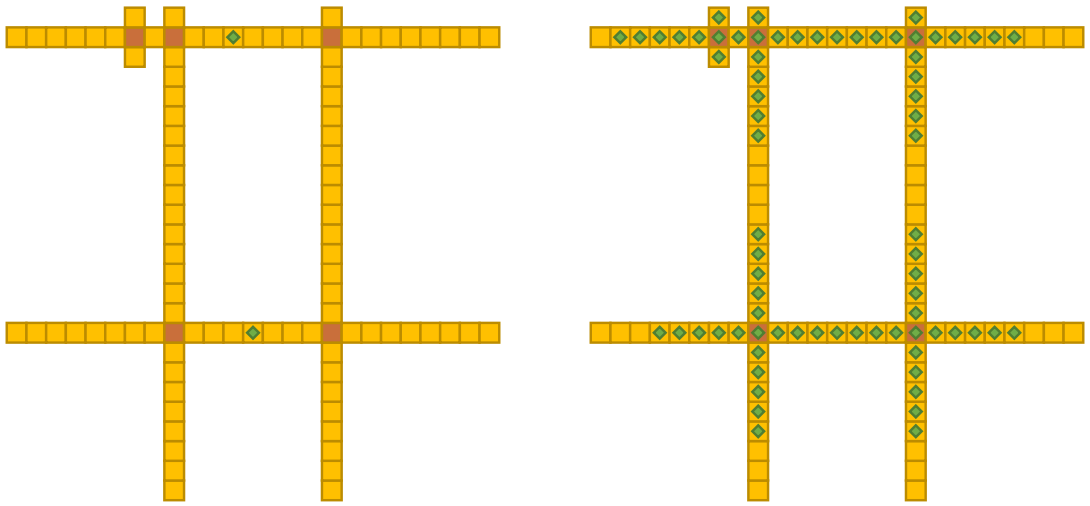
6.3 Greedy heuristics for bounded vision

In the bounded-vision setting, the structure of optimal solutions differs from the unbounded case. When the visibility radius is limited to a fixed value k, it is no longer sufficient to consider only cameras placed at junctions. A junction camera may fail to see gates that lie beyond distance k along a street.

Therefore, an optimal solution may require cameras positioned away from junctions. More precisely, for each junction it is necessary to consider candidate camera locations that lie at distance at most k from that junction along the streets. These locations

can provide coverage that junction cameras cannot achieve under the bounded visibility constraint (Figure 6.3).

Consequently, in the bounded-vision model the search space for optimal camera placements expands from junctions alone to all feasible positions within distance k from junctions. This fundamental difference explains why the matrix representation is used and why optimal solutions may include non-junction camera placements.



(a) (b)

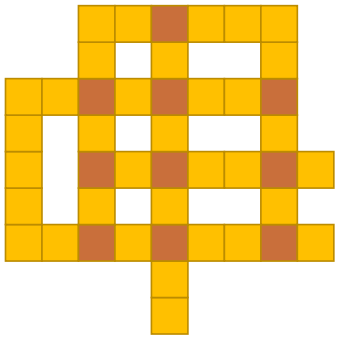
Figure 6.3: (a) An optimal solution for bounded visibility with $k = 5$. (b) Candidate camera locations considered for obtaining the optimal solution.

6.4 Cities (Types of thin polyominoes)

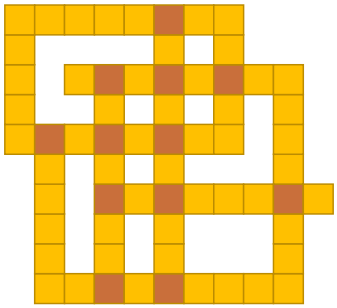
The experimental evaluation is conducted on a diverse collection of city configurations, each modeled as a thin polyomino (see Fig. 6.4). These instances are designed to capture a wide range of structural properties that commonly arise in both synthetic and practical settings, and to stress different aspects of the proposed algorithms.

1. *Polyominoes with a controlled number of intersections*, allowing precise regulation of junction density while keeping the overall shape fixed.
2. *Polyominoes with a controlled number of holes*, used to evaluate the impact of internal obstacles and nontrivial topology on algorithmic performance.
3. *Polyominoes composed solely of square junctions*, representing highly regular city layouts with uniform local structure.
4. *Polyominoes composed solely of T-junctions*, capturing street networks with asymmetric branching and restricted visibility patterns.
5. *Regular randomly generated polyominoes*, serving as unbiased baseline instances with no enforced structural constraints.
6. *Tree-shaped polyominoes*, in which the underlying street graph is acyclic, allowing the behavior of the algorithms to be examined in the absence of cycles.

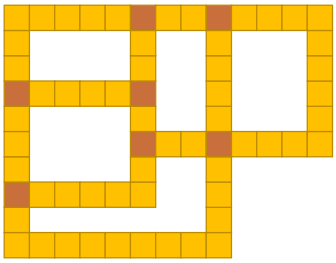
Together, these instance families provide a comprehensive benchmark for evaluating approximation quality and robustness across markedly different classes of thin polyominoes.



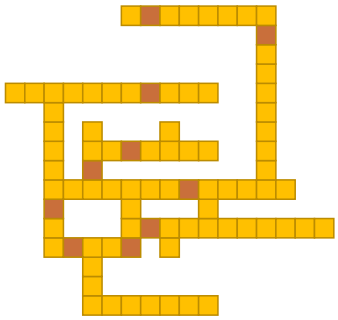
(a) Controlled number of intersections (10), gates on junctions.



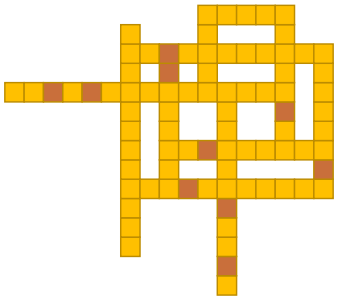
(b) Controlled number of holes within the thin polyomino (9), gates on junctions.



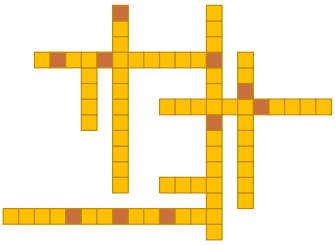
(c) Polyominoes composed solely of squares (no free line crossing), gates on junctions.



(d) Polyominoes composed solely of T-junctions, Random gates position.



(e) Regular randomly generated polyominoes, Random gates position.



(f) Trees, Random gates position.

Figure 6.4: Types of cities and gates spreading.

Each series of city instances is generated with progressively increasing size, allowing the scalability of the algorithms to be evaluated in a controlled manner. Two primary gate-placement scenarios are considered. In the first scenario, all gates are located exclusively at junctions. In the second scenario, gates are distributed randomly throughout the city, including positions along streets and not necessarily at junctions, with the distribution applied to increasingly larger instances.

Dominating cameras

A direct implementation would consider all unit squares of P as potential camera locations. This is unnecessary. A camera c is called *dominating* if there is no other camera c' such that $\mathrm{Vis}(c) \subseteq \mathrm{Vis}(c')$. Only dominating cameras need to be considered, since any non-dominating camera can be replaced without loss by a camera that sees at least the same set of gates. Dominating cameras occur only at gates themselves or at junctions that see at least two gates. The total number of such candidates is linear in

$m+|\mathcal{X}|$. Maintaining the sets $\mathrm{Vis}(c)$ in a priority structure yields an overall running time of $O(m + |\mathcal{X}| \log |\mathcal{X}|)$.

6.5 Unbounded vision results

Recall that $\mathcal{G}^{\mathcal{X}} = ((H \cup \hat{H}) \cup (V \cup \hat{V}), E^{\mathcal{X}})$ is the bipartite graph defined in Section 5.1 (see also Figure 4.2b), where vertices in $H \cup \hat{H}$ and $V \cup \hat{V}$ represent horizontal and vertical streets (including dummy streets), respectively, and each edge in $E^{\mathcal{X}}$ corresponds to exactly one gate in $\mathcal{X}$. An edge $\{h, v\} \in E^{\mathcal{X}}$ is marked red if the corresponding gate must be covered by the selected solution. The goal of the algorithms presented in this section is therefore to select a minimum set of edges of $\mathcal{G}^{\mathcal{X}}$ that dominates all red edges, that is, such that every red edge shares at least one endpoint with a selected edge. The algorithms presented select edges according to various strategies, repeatedly removing endpoints and incident edges from the graph until no red edges remain.

Algorithm 1: Random Selection

This heuristic serves as a baseline: it selects cameras without using any structural information about the graph. Its purpose is to measure how much improvement is obtained by the more informed greedy strategies compared with a purely random maximal edge-dominating construction.

1. Choose uniformly at random an edge $e \in E$.
2. Add e to the solution set.
3. Remove endpoints of e and all incident edges from G.
4. Repeat steps 1–3 until no red edges remain.

Algorithm 2: Random Greedy Selection

This heuristic follows the natural greedy principle of choosing, at each step, an edge that dominates as many currently uncovered red edges as possible. The idea is that selecting such an edge should rapidly reduce the number of remaining gates, producing a smaller residual instance after each iteration.

1. Select the edge $e \in E$ with the maximum number of adjacent red edges (ties broken uniformly at random).
2. Add e to the solution set.
3. Remove endpoints of e and all incident edges from G.
4. Repeat steps 1–3 until no red edges remain.

Algorithm 3: Necessary-First Selection

This heuristic gives priority to red leaf edges, since a red edge incident to a leaf vertex has very limited possibilities for being dominated. By selecting a parent edge first, the algorithm covers such forced or nearly forced gates early, while possibly covering

additional red edges incident to the parent endpoint. After these necessary choices are exhausted, the remaining instance is handled by the greedy rule of Algorithm 2.

1. Identify red leaf edges in G. A red leaf edge is a red edge $e = \{u, v\}$ such that at least one of its endpoints has degree one in G.
2. If leaf edges exist, add their parent edges to the solution. If no leaf edge exists, use Random Greedy Selection (Algorithm 2).
3. Remove endpoints of the selected edge and all incident edges from G.
4. Repeat steps 1–3 until no red edges remain.

Algorithm 4: Minimum Visible Gate Selection

This heuristic first focuses on a red edge that has the fewest adjacent red edges, and is therefore among the most constrained gates in the current residual graph. Once such a gate is identified, the algorithm chooses a neighboring edge that dominates as many red edges as possible, combining a "handle the difficult gates first" rule with a greedy coverage rule.

1. Select a red edge $e \in E$ with the minimum number of adjacent red edges.
2. Choose an edge adjacent to e that has the maximum number of adjacent red edges,(ties broken uniformly at random).
3. Add the chosen edge to the solution set.
4. Remove endpoints of the chosen edge and all incident edges from G.
5. Repeat steps 1–4 until no red edges remain.

Algorithm 5: Minimum Adjacent-to-Adjacent Selection

This heuristic uses a one-step lookahead. It first identifies a highly connected red edge, and then compares its neighboring candidate edges according to the size of their surrounding neighborhoods. Choosing the candidate with the minimum adjacent-to-adjacent sum is intended to avoid spending a camera on an overly redundant region, while still ensuring that the selected red edge is covered.

1. Select a red edge $e \in E$ with the maximum number of adjacent edges.
2. For each neighbor edge of e, count the total number of its neighbors, and compute how many are they - this is the sum.
3. Choose the edge with the minimum sum from step 2 (ties broken uniformly at random).
4. Add the chosen edge to the solution set.
5. Remove endpoints of the chosen edge and all incident edges from G.
6. Repeat steps 1–5 until no red edges remain.

Since the algorithms under consideration exhibit inherent randomness, multiple independent executions are performed for each configuration. For instances in which all gates are located at junctions, each algorithm is executed 1000 times. For instances with randomly placed gates, each algorithm is executed 2500 times. For every city instance, the

minimum number of cameras obtained across all executions is reported, corresponding to the best solution found for that instance.

The solutions obtained by the proposed algorithms are evaluated against optimal solutions computed using 0–1 linear programming formulations. For each instance, the approximation ratio ALG/OPT is reported, and detailed numerical results are summarized in Tables 6.1 and 6.2 for the two gate-placement scenarios.

Conclusions

The experimental results indicate that naive random edge selection yields consistently weak performance, often producing solutions far from optimal. In contrast, greedy algorithms equipped with structured tie-breaking heuristics, most notably Algorithms 3-5 exhibit robust behavior across all tested classes of cities. These heuristics repeatedly generate solutions that are close to optimal, independent of the underlying polyomino structure.

Overall, although greedy strategies admit unfavorable worst-case guarantees in theory, the empirical evidence demonstrates that, when combined with appropriate heuristics, they provide highly effective and scalable solutions for realistic city instances.

	$\frac{\text{avg(Algo 1)}}{\text{01LP}}$	$\frac{\text{avg(Algo 2)}}{\text{01LP}}$	$\frac{\text{avg(Algo 3)}}{\text{01LP}}$	$\frac{\text{avg(Algo 4)}}{\text{01LP}}$	$\frac{\text{avg(Algo 5)}}{\text{01LP}}$
Controlled number of intersections	1.3500	1.2000	1.1250	1.1250	1.1250
Controlled number of holes	1.3158	1.1875	1.1875	1.1875	1.1875
Polyominoes composed solely of squares	1.2857	1.2143	1.2143	1.2143	1.2143
Polyominoes composed solely of T-junctions	1.3519	1.1739	1.1282	1.1282	1.1282
Regular randomly generated polyominoes	1.2857	1.2000	1.1429	1.1429	1.1429
Trees	1.2000	1.2000	1.2000	1.0000	1.0000

Table 6.1: Results for cities with gates at junctions

	$\frac{\text{avg(Algo 1)}}{\text{01LP}}$	$\frac{\text{avg(Algo 2)}}{\text{01LP}}$	$\frac{\text{avg(Algo 3)}}{\text{01LP}}$	$\frac{\text{avg(Algo 4)}}{\text{01LP}}$	$\frac{\text{avg(Algo 5)}}{\text{01LP}}$
Controlled number of intersections	1.7931	1.1154	1.1154	1.1154	1.1154
Controlled number of holes	1.8571	1.1429	1.1429	1.0930	1.0930
Polyominoes composed solely of squares	1.8571	1.2500	1.0714	1.0500	1.0455
Polyominoes composed solely of T-junctions	1.6438	1.1429	1.1429	1.0833	1.0833
Regular randomly generated polyominoes	1.6364	1.0909	1.0909	1.0870	1.0870
Trees	1.5714	1.1250	1.1250	1.0625	1.0625

Table 6.2: Results for cities with randomly distributed gates

Detailed experimental results are presented in Appendix A. In addition, all experiments can be reproduced and extended using the publicly available web-based simulator, accessible at this link [42]. The interface allows interactive exploration of different city configurations, gate distributions, and algorithmic parameters.

6.6 Bounded vision results

In this section, algorithms are described for the bounded-vision variant of the gate-cover problem. Unlike the unbounded-vision setting, the algorithms operate directly on a binary *gate-camera matrix* rather than on a bipartite graph representation.

In the bounded model, cameras are not restricted to junctions. As discussed earlier, when the visibility radius is limited to a fixed value k, an optimal solution may require cameras positioned away from junctions. In particular, for every junction and for every street incident to it, all feasible camera locations at distance at most k from that junction are considered as candidate positions. These positions form the set of potential camera locations used to construct the matrix.

For a fixed vision parameter k, let $A \in \{0,1\}^{m \times n}$ denote the gate-camera matrix, where rows correspond to gates and columns correspond to candidate camera locations determined by the above rule. An entry $A_{i,j} = 1$ if and only if camera j sees gate i under the k-bounded visibility model. Thus, selecting a camera corresponds to selecting a column of A, and covering a gate corresponds to covering a row that contains at least one selected column with value 1.

All algorithms repeatedly select camera columns according to different selection rules. Whenever a camera j is selected, all rows i with $A_{i,j} = 1$ are considered covered and are removed from further consideration by zeroing the corresponding rows in the matrix. The process terminates once no uncovered rows remain.

This formulation reduces the bounded-vision gate-cover problem to a structured set-cover instance, where the geometry of the polyomino is encoded entirely in the matrix A.

Algorithm 1: Random Column Selection

This heuristic serves as a baseline for the matrix formulation. It ignores the structure of the instance and repeatedly selects a feasible column at random, thereby producing a simple randomized set-cover construction against which the more informed heuristics can be compared.

1. Choose uniformly at random a column j such that it covers at least one uncovered row.
2. Add j to the solution set.
3. Remove all rows covered by column j from the matrix.
4. Repeat steps 1-3 until all rows are covered.

Algorithm 2: Greedy Maximum Cover

This heuristic follows the standard greedy approach for set cover. At each step, it selects a column that covers the largest number of currently uncovered rows, with the goal of reducing the residual instance as quickly as possible.

1. For each column, compute the number of currently uncovered rows it covers.
2. Select a column with maximum coverage (ties broken uniformly at random).
3. Add the column to the solution set.
4. Remove all rows covered by the selected column.
5. Repeat steps 1-4 until all rows are covered.

Algorithm 3: Necessary-First Selection

This heuristic first handles rows that are covered by exactly one column. Since the unique column covering such a row must appear in any feasible solution, selecting it early is natural; moreover, this column may also cover additional rows, leaving a smaller residual problem for the greedy rule.

1. Identify rows that are covered by exactly one column (necessary rows).
2. If such rows exist, select the unique covering column of one of them (chosen uniformly at random) and add it to the solution.
3. Otherwise, apply Greedy Maximum Cover (Algorithm 2).
4. Remove all rows covered by the selected column.
5. Repeat steps 1-4 until all rows are covered.

Algorithm 4: Greedy Maximum with Minimum Conflict

This heuristic refines the greedy maximum-cover rule by adding a tie-breaking criterion based on conflict. Among the columns that cover the maximum number of uncovered rows, it prefers a column whose covered rows have fewer alternative coverings, so that constrained rows are handled early while still maintaining high immediate coverage.

1. For each column, compute its current coverage (number of uncovered rows it covers).
2. Restrict attention to columns with maximum coverage.
3. Among these columns, compute for each the total number of additional potential coverings of its covered rows (a conflict measure).
4. Select a column with minimum conflict (ties broken uniformly at random).
5. Add the column to the solution set.
6. Remove all rows covered by the selected column.
7. Repeat steps 1-6 until all rows are covered.

Since the algorithms exhibit inherent randomness, 500 independent executions are performed for each configuration.

We conduct two types of experiments. In the first type, for the bounded-vision experiment with fixed $k = 5$, each algorithm is executed repeatedly for every city instance, and the minimum number of cameras obtained across all executions is reported. Optimal solutions are computed using $0-1$ linear programming formulations applied to the same gate-camera matrix.

In a second experiment, the number of gates is kept fixed while the visibility parameter varies. Specifically, $k \in \{5, 7, 10, 13, 16\}$. For each value of k, a new gate-camera matrix is constructed reflecting the corresponding bounded visibility radius. The same four algorithms are applied independently to each matrix, and their performance is compared against the optimal solution.

For every instance and every value of k, the approximation ratio ALG/OPT is reported. Detailed numerical results are summarized in Table 6.3 for $k = 5$ and in Table 6.4 for the varying k experiment. The raw numerical data are provided in the supplementary material.

Conclusions

The experimental results demonstrate that pure random selection produces unstable and frequently suboptimal solutions. In contrast, structured greedy strategies-particularly the Necessary-First heuristic and the maximum-cover selection with conflict minimization-exhibit robust behavior across all tested values of k. As k increases, visibility improves and the optimal number of cameras decreases, yet the relative ranking of the algorithms remains consistent.

Overall, the bounded-vision variant is essentially a set-cover problem and therefore inherits the usual worst-case theoretical limitations. However, the experimental results show that simple and carefully designed greedy heuristics perform very well in practice, producing near-optimal and scalable solutions for realistic city instances and for different values of the visibility radius.

	$\frac{\text{avg(Algo 1)}}{\text{01LP}}$	$\frac{\text{avg(Algo 2)}}{\text{01LP}}$	$\frac{\text{avg(Algo 3)}}{\text{01LP}}$	$\frac{\text{avg(Algo 4)}}{\text{01LP}}$
Controlled number of intersections	1.8750	1.1429	1.1429	1.1667
Controlled number of holes	2.0000	1.1250	1.1250	1.1667
Polyominoes composed solely of squares	1.8889	1.0833	1.1000	1.1250
Polyominoes composed solely of T-junctions	1.7778	1.2000	1.2000	1.2000
Regular randomly generated polyominoes	2.1111	1.1500	1.1500	1.1750
Trees	2.0000	1.0000	1.2500	1.1818

Table 6.3: Results for bounded visibility with $k = 5$

	$\frac{\text{avg(Algo 1)}}{\text{01LP}}$	$\frac{\text{avg(Algo 2)}}{\text{01LP}}$	$\frac{\text{avg(Algo 3)}}{\text{01LP}}$	$\frac{\text{avg(Algo 4)}}{\text{01LP}}$
Controlled number of intersections	2.2963	1.2000	1.2000	1.3000
Controlled number of holes	3.0000	1.1875	1.1667	1.2308
Polyominoes composed solely of squares	2.5294	1.2222	1.1765	1.2222
Polyominoes composed solely of T-junctions	2.0313	1.2000	1.2000	1.2286
Regular randomly generated polyominoes	2.4737	1.2222	1.1765	1.3333
Trees	2.5000	1.3333	1.3333	1.3333

Table 6.4: Results for varying visibility radius k

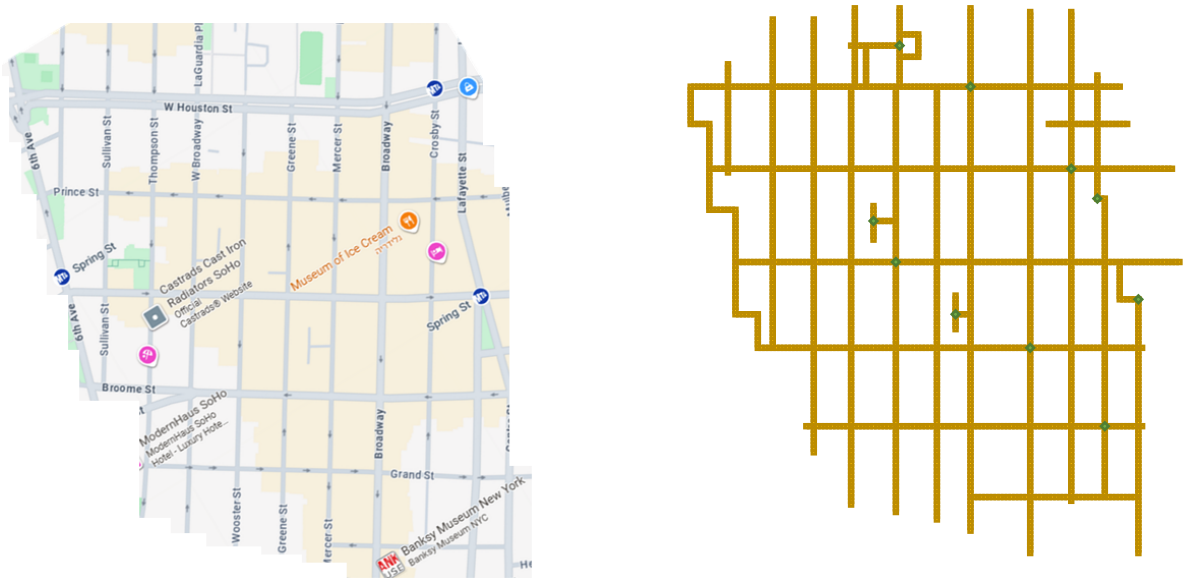
Detailed experimental results are presented in Appendix B.

6.7 Real City Case Study

A natural direction for evaluating the practical relevance of the proposed framework is to examine its behavior on real-world urban layouts. Beyond synthetic and randomly generated instances, actual city structures exhibit irregularities, varying block sizes, and nonuniform street densities that may influence algorithmic performance. To investigate this aspect, a representative portion of Manhattan is selected as a case study (see Figure 6.5(a)). The chosen area captures the characteristic grid structure of the city while including local variations typical of real urban environments.

In this experiment, gates are assumed to be located at all street junctions, reflecting a dense deployment scenario in which every intersection requires coverage. Algorithm 5 is executed more than 10,000 times in order to account for its randomized tie-breaking mechanism and to explore the variability of the solutions produced. For each run, the number of cameras selected is recorded, and the best solution obtained over all executions is retained.

The smallest solution identified requires only 10 cameras, as illustrated in Figure 6.5(b). Moreover, even the least favorable outcome observed during the experiments required no more than 11 cameras. The narrow range between the best and worst solutions indicates a high degree of stability in the algorithm's behavior. Although optimality is not formally certified in this instance, the consistently small gap suggests that the solutions produced are very close to optimal.



(a) Small subsection of Manhattan.

(b) Real city results of Algo-5.

Figure 6.5: Real city

This case study highlights two important aspects. First, the modeling framework naturally accommodates realistic urban topologies without requiring structural simplifications. Second, the heuristic approach demonstrates strong empirical performance even in complex, nonuniform city layouts. The results indicate that the algorithm remains effective beyond idealized thin polyomino models, providing near-optimal solutions in practical settings.

7 Conclusions

This thesis introduced and studied the *Gate-Cover Problem in Thin Polyominoes*, a guarding problem motivated by monitoring and localization in urban-like networks. Unlike the classical Art Gallery Problem, the objective is not to guard the entire domain, but only a designated set of gates using cameras whose visibility is restricted by the street structure of a thin polyomino. The work developed both theoretical and experimental foundations for this new problem under unbounded and bounded visibility models.

We introduced the motivation for the Gate-Cover Problem and placed it within the broader context of the Art Gallery Problem and its variants. We explained why covering selected gates rather than the entire region leads to a different combinatorial structure, and presented the main contributions of the thesis: hardness results, approximation algorithms, structural results, and experimental evaluation.

We formally defined the Gate-Cover Problem. It introduced the city model based on thin polyominoes, described streets, gates, and camera locations, and defined the two visibility settings considered throughout the thesis: unbounded visibility and k-bounded visibility. We provided the precise mathematical framework used in the subsequent hardness, algorithmic, and experimental analysis.

We established the computational hardness of the Gate-Cover Problem. For the unbounded-visibility model, the problem was related to edge domination in bipartite grid intersection graphs, and NP-hardness was shown even under strong restrictions, including instances formed by T-junctions. For the bounded-visibility model, NP-hardness was proved for every fixed visibility range $k \geq 1$, demonstrating that limiting the camera range does not make the problem computationally easy in general.

We developed positive algorithmic results that complement the hardness results. For the unbounded model, the thesis presented a near-linear-time 2-approximation algorithm based on the connection to edge domination. Also, we showed that additional structure can be exploited: bounded-treewidth thin polyominoes admit dynamic-programming approaches, separator methods yield exact subexponential algorithms, and the bounded-visibility variant admits a PTAS for every constant range k.

We studied the practical performance of greedy and heuristic algorithms for both visibility models. First, we showed that simple greedy methods can have unfavorable worst-case behavior, including approximation ratios approaching 2, and then introduced refined heuristics designed to improve performance in practice. The experimental results show that these heuristics often produce solutions close to optimal across many synthetic city types and in a real-city case study, especially when compared with optimal solutions computed by 0-1 linear programming.

Bridging Theory and Practice. Taken together, the results of this thesis show that the Gate-Cover Problem is theoretically challenging but algorithmically meaningful. The hardness results establish that the problem is intractable in general, while the approx-

imation, treewidth, separator, and PTAS results show that useful structure can still be exploited. The experimental evaluation further supports this conclusion: although worst-case guarantees may be pessimistic, carefully designed heuristics provide effective and scalable solutions for realistic instances.

Overall, this thesis contributes a new guarding model that connects computational geometry, graph domination, approximation algorithms, and practical surveillance applications. It shows that thin polyominoes provide a rich framework for studying localized monitoring problems, where full visibility is unnecessary and the goal is instead to cover strategically important gates. Several natural directions remain open, including improving approximation guarantees for special subclasses, developing stronger parameterized algorithms, and extending the experimental framework to larger and more realistic urban networks.

Appendix

Appendix A: Detailed Experimental Results: Unbounded Vision

A.1 Detailed results for the cases involving gates placed on junctions

City Type	city size (# of cells in poly-omino)	# of gates	Algo1/01LP	Algo2/01LP	Algo3/01LP	Algo4/01LP	Algo5/01LP
Controlled number of intersections	18	1	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	22	2	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	29	5	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	45	7	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	44	10	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	56	13	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	103	17	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	107	20	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	164	26	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	229	30	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	336	36	1.1667	1.1667	1.0000	1.0000	1.0000
Controlled number of intersections	407	41	1.1250	1.1250	1.1250	1.1250	1.1250
Controlled number of intersections	428	46	1.1000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	512	52	1.2500	1.1250	1.0000	1.0000	1.0000
Controlled number of intersections	654	62	1.2000	1.2000	1.1000	1.1000	1.1000
Controlled number of intersections	905	71	1.3000	1.1000	1.1000	1.1000	1.1000
Controlled number of intersections	1084	79	1.2143	1.0714	1.0714	1.0714	1.0714
Controlled number of intersections	1075	87	1.1429	1.0714	1.0714	1.0714	1.0714
Controlled number of intersections	1071	96	1.2500	1.0833	1.0833	1.0833	1.0833
Controlled number of intersections	1575	122	1.1875	1.0625	1.0625	1.0625	1.0625
Controlled number of intersections	2042	142	1.2381	1.0476	1.0476	1.0476	1.0476
Controlled number of intersections	2269	165	1.2273	1.0909	1.0909	1.0909	1.0909
Controlled number of intersections	2621	179	1.3333	1.0833	1.0417	1.0417	1.0417
Controlled number of intersections	3168	190	1.2800	1.1200	1.1200	1.1200	1.1200
Controlled number of intersections	3924	231	1.2581	1.0323	1.0323	1.0323	1.0323
Controlled number of intersections	4356	250	1.3030	1.1212	1.1212	1.1212	1.1212
Controlled number of intersections	5140	283	1.2647	1.0588	1.0588	1.0588	1.0588
Controlled number of intersections	5541	299	1.3056	1.1111	1.0833	1.0833	1.0833
Controlled number of intersections	7451	344	1.3500	1.1250	1.1000	1.1000	1.1000
Controlled number of intersections	9639	395	1.2979	1.0851	1.0851	1.0851	1.0851
Controlled number of holes	22	2	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	16	2	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	22	5	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	34	6	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	64	12	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	111	15	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	152	25	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	235	42	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	336	45	1.1429	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	361	63	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	475	62	1.1429	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	621	74	1.0909	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	511	71	1.2222	1.1111	1.1111	1.1111	1.1111
Controlled number of holes	772	79	1.0833	1.0833	1.0833	1.0833	1.0833
Controlled number of holes	597	83	1.2000	1.1000	1.1000	1.1000	1.1000
Controlled number of holes	868	94	1.1818	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	1005	98	1.2143	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	866	94	1.2143	1.0714	1.0714	1.0714	1.0714
Controlled number of holes	923	108	1.1818	1.1818	1.1818	1.1818	1.1818
Controlled number of holes	1184	123	1.1875	1.1250	1.1250	1.1250	1.1250
Controlled number of holes	1481	126	1.2667	1.0667	1.0667	1.0667	1.0667
Controlled number of holes	1254	124	1.2000	1.0667	1.0667	1.0667	1.0667
Controlled number of holes	1521	133	1.2500	1.1875	1.1875	1.1875	1.1875
Controlled number of holes	1484	142	1.2632	1.0526	1.0526	1.0526	1.0526
Controlled number of holes	1767	151	1.2632	1.1053	1.1053	1.1053	1.1053
Controlled number of holes	1857	158	1.2857	1.0952	1.0952	1.0952	1.0952
Controlled number of holes	1974	163	1.2500	1.0500	1.0500	1.0500	1.0500
Controlled number of holes	2308	173	1.3158	1.1579	1.1579	1.1579	1.1579
Controlled number of holes	2299	181	1.2727	1.0455	1.0455	1.0455	1.0455

Controlled number of holes	2302	204	1.2632	1.1579	1.1579	1.1579	1.1579
Polyominoes composed solely of squares	12	2	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	16	4	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	59	6	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	63	7	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	100	10	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	124	15	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	237	22	1.2000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	309	34	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	315	26	1.2000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	329	24	1.2000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	354	32	1.2000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	452	41	1.1429	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	539	51	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	649	65	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	652	67	1.1818	1.0909	1.0909	1.0909	1.0909
Polyominoes composed solely of squares	796	71	1.1111	1.1111	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	827	74	1.2000	1.1000	1.1000	1.1000	1.1000
Polyominoes composed solely of squares	880	84	1.1000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1007	108	1.2727	1.0909	1.0909	1.0909	1.0909
Polyominoes composed solely of squares	1008	92	1.2000	1.1000	1.1000	1.1000	1.1000
Polyominoes composed solely of squares	1107	93	1.1538	1.0769	1.0769	1.0769	1.0769
Polyominoes composed solely of squares	1140	70	1.1818	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1143	83	1.1667	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1222	132	1.2308	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1324	130	1.2000	1.2000	1.2000	1.2000	1.2000
Polyominoes composed solely of squares	1450	112	1.1538	1.0769	1.0769	1.0769	1.0769
Polyominoes composed solely of squares	1553	124	1.2857	1.2143	1.2143	1.2143	1.2143
Polyominoes composed solely of squares	1682	133	1.2000	1.0667	1.0667	1.0667	1.0667
Polyominoes composed solely of squares	1700	141	1.2500	1.1667	1.1667	1.1667	1.1667
Polyominoes composed solely of squares	1750	145	1.2000	1.0667	1.0667	1.0667	1.0667
Polyominoes composed solely of T-junctions	17	1	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	18	2	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	84	12	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	87	11	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	124	17	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	155	24	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	181	28	1.3333	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	300	43	1.0909	1.0909	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	302	52	1.2308	1.0769	1.0769	1.0769	1.0769
Polyominoes composed solely of T-junctions	341	51	1.1667	1.1667	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	426	57	1.2857	1.0714	1.0714	1.0714	1.0714
Polyominoes composed solely of T-junctions	520	66	1.1875	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	617	99	1.3000	1.0500	1.0500	1.0500	1.0500
Polyominoes composed solely of T-junctions	638	86	1.1429	1.0952	1.0476	1.0476	1.0476
Polyominoes composed solely of T-junctions	694	103	1.2174	1.1739	1.0435	1.0435	1.0435
Polyominoes composed solely of T-junctions	776	100	1.1600	1.0800	1.0400	1.0400	1.0400
Polyominoes composed solely of T-junctions	802	119	1.2308	1.0769	1.0385	1.0385	1.0385
Polyominoes composed solely of T-junctions	974	145	1.1875	1.0625	1.0625	1.0625	1.0625
Polyominoes composed solely of T-junctions	1049	144	1.2857	1.0571	1.0571	1.0571	1.0571
Polyominoes composed solely of T-junctions	1067	149	1.2500	1.0556	1.0556	1.0556	1.0556
Polyominoes composed solely of T-junctions	1125	148	1.3226	1.0968	1.0968	1.0968	1.0968
Polyominoes composed solely of T-junctions	1128	164	1.2353	1.0294	1.0294	1.0294	1.0294
Polyominoes composed solely of T-junctions	1248	178	1.3235	1.0294	1.0294	1.0294	1.0294
Polyominoes composed solely of T-junctions	1252	185	1.2821	1.1282	1.1282	1.1282	1.1282
Polyominoes composed solely of T-junctions	1488	185	1.3171	1.0488	1.0488	1.0488	1.0488
Polyominoes composed solely of T-junctions	1520	193	1.3095	1.1429	1.1190	1.1190	1.1190
Polyominoes composed solely of T-junctions	1558	203	1.2553	1.1064	1.0638	1.0638	1.0638
Polyominoes composed solely of T-junctions	1612	198	1.2727	1.0682	1.0682	1.0682	1.0682
Polyominoes composed solely of T-junctions	1683	219	1.2800	1.1200	1.1000	1.1000	1.1000
Polyominoes composed solely of T-junctions	1840	246	1.3519	1.0741	1.0741	1.0741	1.0741
Regular randomly generated polyominoes	18	1	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	22	2	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	83	16	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	122	18	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	150	20	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	161	28	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	233	36	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	339	65	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	397	62	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	469	71	1.1111	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	535	84	1.1111	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	560	82	1.0909	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	613	103	1.2000	1.2000	1.1000	1.1000	1.1000
Regular randomly generated polyominoes	785	108	1.1000	1.1000	1.1000	1.0000	1.0000
Regular randomly generated polyominoes	874	129	1.0909	1.0909	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	945	157	1.1818	1.0909	1.0909	1.0909	1.0909
Regular randomly generated polyominoes	1116	176	1.0769	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	1159	184	1.1333	1.0667	1.0667	1.0667	1.0667
Regular randomly generated polyominoes	1185	165	1.1538	1.0769	1.0769	1.0769	1.0769
Regular randomly generated polyominoes	1375	225	1.2143	1.1429	1.1429	1.1429	1.1429
Regular randomly generated polyominoes	1426	202	1.1429	1.0714	1.0714	1.0714	1.0714
Regular randomly generated polyominoes	1489	231	1.2500	1.1875	1.1250	1.1250	1.1250
Regular randomly generated polyominoes	1620	231	1.1579	1.0526	1.0526	1.0526	1.0526
Regular randomly generated polyominoes	1649	241	1.1667	1.1111	1.0556	1.0556	1.0556
Regular randomly generated polyominoes	1762	283	1.2000	1.1000	1.1000	1.1000	1.1000
Regular randomly generated polyominoes	2091	301	1.2000	1.1000	1.1000	1.1000	1.1000
Regular randomly generated polyominoes	2155	322	1.2105	1.1053	1.0526	1.0526	1.0526
Regular randomly generated polyominoes	2188	277	1.2500	1.1000	1.0500	1.0500	1.0500
Regular randomly generated polyominoes	2317	344	1.1905	1.0952	1.0952	1.0952	1.0952
Regular randomly generated polyominoes	2504	374	1.2857	1.1905	1.1429	1.1429	1.1429
Tree	6	3	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	8	5	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	43	4	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	44	5	1.0000	1.0000	1.0000	1.0000	1.0000

Tree	74	6	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	76	7	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	82	7	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	113	11	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	124	7	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	146	12	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	165	11	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	189	13	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	262	15	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	359	19	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	361	20	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	362	19	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	381	22	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	448	23	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	456	26	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	521	24	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	524	25	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	559	31	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	560	26	1.2000	1.2000	1.2000	1.0000	1.0000
Tree	569	27	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	601	31	1.1429	1.1429	1.1429	1.0000	1.0000
Tree	722	27	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	3011	53	1.1000	1.1000	1.1000	1.0000	1.0000
Tree	4241	59	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	4998	64	1.0714	1.0714	1.0714	1.0000	1.0000
Tree	6849	75	1.0000	1.0000	1.0000	1.0000	1.0000

A.2 Detailed results for the cases where N gates are distributed randomly

City Type	city size (# of cells in poly-omino)	# of gates	Algo1/01LP	Algo2/01LP	Algo3/01LP	Algo4/01LP	Algo5/01LP
Controlled number of intersections	29	1	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	29	2	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	29	4	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	29	5	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	29	7	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	107	5	1.3333	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	107	10	1.7500	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	107	16	1.2500	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	107	21	1.2500	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	107	26	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	428	21	1.3333	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	428	42	1.6154	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	428	64	1.3529	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	428	85	1.3684	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	428	107	1.4118	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	1071	53	1.4737	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	1071	107	1.5385	1.1154	1.1154	1.1154	1.1154
Controlled number of intersections	1071	160	1.4231	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	1071	214	1.5172	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	1071	267	1.4444	1.0741	1.0741	1.0741	1.0741
Controlled number of intersections	3924	196	1.7000	1.0600	1.0400	1.0400	1.0400
Controlled number of intersections	3924	392	1.7288	1.0508	1.0508	1.0508	1.0169
Controlled number of intersections	3924	588	1.5806	1.0323	1.0323	1.0323	1.0323
Controlled number of intersections	3924	784	1.5758	1.0758	1.0758	1.0758	1.0758
Controlled number of intersections	3924	981	1.5077	1.0462	1.0462	1.0462	1.0462
Controlled number of intersections	9639	481	1.7931	1.0805	1.0805	1.0805	1.0230
Controlled number of intersections	9639	963	1.6900	1.1000	1.1000	1.1000	1.0800
Controlled number of intersections	9639	1445	1.6190	1.0952	1.0952	1.0952	1.0667
Controlled number of intersections	9639	1927	1.5463	1.0833	1.0833	1.0833	1.0556
Controlled number of intersections	9639	2409	1.5278	1.1111	1.1111	1.1111	1.1019
Controlled number of holes	22	1	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	22	2	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	22	3	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	22	4	1.5000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	22	5	1.0000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	235	11	1.2000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	235	23	1.2500	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	235	35	1.2727	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	235	47	1.3636	1.0909	1.0000	1.0000	1.0000
Controlled number of holes	235	58	1.5000	1.1000	1.1000	1.0000	1.0000
Controlled number of holes	511	25	1.8571	1.1429	1.1429	1.0000	1.0000
Controlled number of holes	511	51	1.5714	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	511	76	1.4375	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	511	102	1.4000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	511	127	1.3333	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	923	46	1.6000	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	923	92	1.5556	1.0556	1.0556	1.0556	1.0556
Controlled number of holes	923	138	1.3913	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	923	184	1.4348	1.0435	1.0435	1.0435	1.0435
Controlled number of holes	923	230	1.4545	1.0455	1.0455	1.0455	1.0455
Controlled number of holes	1767	88	1.5185	1.0370	1.0370	1.0370	1.0370
Controlled number of holes	1767	176	1.6765	1.0588	1.0588	1.0588	1.0588
Controlled number of holes	1767	265	1.6842	1.0789	1.0789	1.0789	1.0789
Controlled number of holes	1767	353	1.4667	1.0222	1.0222	1.0222	1.0222
Controlled number of holes	1767	441	1.5116	1.0930	1.0930	1.0930	1.0930
Controlled number of holes	2302	115	1.6286	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	2302	230	1.6154	1.0513	1.0513	1.0513	1.0513
Controlled number of holes	2302	345	1.5000	1.0227	1.0000	1.0000	1.0000
Controlled number of holes	2302	460	1.4043	1.0851	1.0638	1.0638	1.0638
Controlled number of holes	2302	575	1.4889	1.0889	1.0889	1.0889	1.0889
Polyominoes composed solely of squares	59	2	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	59	5	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	59	8	1.3333	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	59	11	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	59	14	1.5000	1.2500	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	309	15	1.2000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	309	30	1.8571	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	309	46	1.3000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	309	61	1.4545	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	309	77	1.2857	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	539	26	1.4545	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	539	53	1.4000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	539	80	1.3571	1.0714	1.0714	1.0000	1.0000
Polyominoes composed solely of squares	539	107	1.3529	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	539	134	1.4118	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1007	50	1.5385	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1007	100	1.5000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1007	151	1.3913	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1007	201	1.4348	1.0435	1.0435	1.0435	1.0435
Polyominoes composed solely of squares	1007	251	1.5000	1.0417	1.0417	1.0417	1.0417
Polyominoes composed solely of squares	1324	66	1.5263	1.0000	1.0000	1.0000	1.0000

Polyominoes composed solely of squares	1324	132	1.6000	1.0500	1.0500	1.0500	1.0000
Polyominoes composed solely of squares	1324	198	1.4615	1.0385	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1324	264	1.5000	1.0455	1.0455	1.0455	1.0455
Polyominoes composed solely of squares	1324	331	1.3704	1.0741	1.0370	1.0370	1.0370
Polyominoes composed solely of squares	1750	87	1.5909	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1750	175	1.3793	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1750	262	1.5161	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	1750	350	1.4667	1.0333	1.0333	1.0333	1.0333
Polyominoes composed solely of squares	1750	437	1.3871	1.0323	1.0323	1.0323	1.0323
Polyominoes composed solely of T-junctions	84	4	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	84	8	1.5000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	84	12	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	84	16	1.2000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	84	21	1.0000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	300	15	1.3750	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	300	30	1.3333	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	300	45	1.2857	1.1429	1.1429	1.0714	1.0714
Polyominoes composed solely of T-junctions	300	60	1.5333	1.0667	1.0667	1.0667	1.0667
Polyominoes composed solely of T-junctions	300	75	1.3333	1.0667	1.0667	1.0667	1.0667
Polyominoes composed solely of T-junctions	617	30	1.3529	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	617	61	1.3333	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	617	92	1.6071	1.0357	1.0357	1.0357	1.0357
Polyominoes composed solely of T-junctions	617	123	1.5152	1.0303	1.0303	1.0303	1.0303
Polyominoes composed solely of T-junctions	617	154	1.4722	1.0833	1.0833	1.0833	1.0833
Polyominoes composed solely of T-junctions	1049	52	1.5000	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	1049	104	1.4390	1.0244	1.0244	1.0244	1.0244
Polyominoes composed solely of T-junctions	1049	157	1.3922	1.0588	1.0588	1.0588	1.0588
Polyominoes composed solely of T-junctions	1049	209	1.4444	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	1049	262	1.4737	1.0526	1.0526	1.0526	1.0526
Polyominoes composed solely of T-junctions	1488	74	1.3333	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	1488	148	1.4615	1.0385	1.0385	1.0385	1.0385
Polyominoes composed solely of T-junctions	1488	223	1.4500	1.0333	1.0333	1.0333	1.0333
Polyominoes composed solely of T-junctions	1488	297	1.4658	1.0274	1.0274	1.0274	1.0274
Polyominoes composed solely of T-junctions	1488	372	1.5205	1.0274	1.0274	1.0274	1.0274
Polyominoes composed solely of T-junctions	1840	92	1.4634	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	1840	184	1.5333	1.0333	1.0333	1.0333	1.0333
Polyominoes composed solely of T-junctions	1840	276	1.6438	1.0411	1.0411	1.0411	1.0411
Polyominoes composed solely of T-junctions	1840	368	1.5949	1.0633	1.0633	1.0633	1.0633
Polyominoes composed solely of T-junctions	1840	460	1.5227	1.0682	1.0682	1.0682	1.0682
Regular randomly generated polyominoes	83	4	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	83	8	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	83	12	1.2000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	83	16	1.2000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	83	20	1.0000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	339	16	1.5000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	339	33	1.3333	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	339	50	1.4000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	339	67	1.3636	1.0909	1.0909	1.0000	1.0000
Regular randomly generated polyominoes	339	84	1.2727	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	613	30	1.2727	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	613	61	1.5625	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	613	91	1.5000	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	613	122	1.3889	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	613	153	1.3500	1.0500	1.0500	1.0500	1.0500
Regular randomly generated polyominoes	1185	59	1.5294	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	1185	118	1.6364	1.0455	1.0455	1.0455	1.0455
Regular randomly generated polyominoes	1185	177	1.4783	1.0870	1.0870	1.0870	1.0870
Regular randomly generated polyominoes	1185	237	1.5000	1.0833	1.0833	1.0833	1.0833
Regular randomly generated polyominoes	1185	296	1.4348	1.0870	1.0870	1.0870	1.0870
Regular randomly generated polyominoes	1762	88	1.5385	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	1762	176	1.4063	1.0625	1.0625	1.0625	1.0625
Regular randomly generated polyominoes	1762	264	1.3514	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	1762	352	1.4324	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	1762	440	1.3333	1.0278	1.0278	1.0278	1.0278
Regular randomly generated polyominoes	2504	125	1.6071	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	2504	250	1.4474	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	2504	375	1.4167	1.0278	1.0278	1.0278	1.0278
Regular randomly generated polyominoes	2504	500	1.4390	1.0732	1.0732	1.0732	1.0732
Regular randomly generated polyominoes	2504	626	1.3409	1.0000	1.0000	1.0000	1.0000
Tree	43	2	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	43	4	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	43	6	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	43	8	1.2500	1.0000	1.0000	1.0000	1.0000
Tree	43	10	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	113	5	1.0000	1.0000	1.0000	1.0000	1.0000
Tree	113	11	1.2000	1.0000	1.0000	1.0000	1.0000
Tree	113	16	1.1429	1.0000	1.0000	1.0000	1.0000
Tree	113	22	1.1667	1.0000	1.0000	1.0000	1.0000
Tree	113	28	1.1429	1.0000	1.0000	1.0000	1.0000
Tree	262	13	1.2000	1.0000	1.0000	1.0000	1.0000
Tree	262	26	1.3750	1.0000	1.0000	1.0000	1.0000
Tree	262	39	1.2222	1.1111	1.1111	1.0000	1.0000
Tree	262	52	1.1000	1.0000	1.0000	1.0000	1.0000
Tree	262	65	1.1818	1.0000	1.0000	1.0000	1.0000
Tree	456	22	1.1538	1.0000	1.0000	1.0000	1.0000
Tree	456	45	1.3077	1.0000	1.0000	1.0000	1.0000
Tree	456	68	1.2500	1.0000	1.0000	1.0000	1.0000
Tree	456	91	1.2667	1.0000	1.0000	1.0000	1.0000
Tree	456	114	1.1250	1.0000	1.0000	1.0000	1.0000
Tree	601	30	1.5714	1.0000	1.0000	1.0000	1.0000
Tree	601	60	1.4000	1.0000	1.0000	1.0000	1.0000
Tree	601	90	1.3750	1.1250	1.1250	1.0625	1.0625
Tree	601	120	1.2632	1.1053	1.1053	1.0526	1.0526
Tree	601	150	1.4000	1.0500	1.0500	1.0500	1.0500
Tree	6849	342	1.1346	1.0000	1.0000	1.0000	1.0000
Tree	6849	684	1.1525	1.0169	1.0169	1.0169	1.0169

Tree	6849	1027	1.1228	1.0000	1.0000	1.0000	1.0000
Tree	6849	1369	1.1356	1.0000	1.0000	1.0000	1.0000
Tree	6849	1712	1.1000	1.0333	1.0167	1.0167	1.0167

Appendix B: Detailed Experimental Results: Bounded Vision

B.1 Detailed results for the cases involving $k = 5$

City Type	city size (# of cubic in poly-omino)	# of gates	Algo1/01LP	Algo2/01LP	Algo3/01LP	Algo4/01LP
Controlled number of intersections	1	18	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	2	22	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	5	29	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	7	45	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	10	44	1.3333	1.0000	1.0000	1.0000
Controlled number of intersections	13	56	1.6667	1.0000	1.0000	1.0000
Controlled number of intersections	17	103	1.5000	1.0000	1.0000	1.0000
Controlled number of intersections	20	107	1.5000	1.0000	1.0000	1.1667
Controlled number of intersections	26	164	1.8571	1.1429	1.1429	1.1429
Controlled number of intersections	30	229	1.8750	1.1250	1.1250	1.1250
Controlled number of intersections	36	336	1.8333	1.0833	1.0833	1.0833
Controlled number of intersections	41	407	1.7857	1.0000	1.0000	1.0714
Controlled number of intersections	46	428	1.7500	1.0625	1.1250	1.0625
Controlled number of intersections	52	512	1.8235	1.0588	1.0588	1.0588
Controlled number of intersections	62	654	1.7727	1.0000	1.0455	1.0000
Controlled number of intersections	71	905	1.5185	1.0370	1.0370	1.1111
Controlled number of intersections	79	1084	1.5625	1.0625	1.0938	1.0625
Controlled number of intersections	87	1075	1.7879	1.0000	1.0303	1.0606
Controlled number of intersections	96	1071	1.8125	1.0000	1.0313	1.0313
Controlled number of intersections	122	1575	1.7111	1.0444	1.0667	1.0444
Controlled number of intersections	142	2042	1.7736	1.0189	1.0189	1.0377
Controlled number of intersections	165	2269	1.7143	1.0000	1.0159	1.0159
Controlled number of intersections	179	2621	1.6892	1.0135	1.0270	1.0270
Controlled number of intersections	190	3168	1.7125	1.0125	1.0375	1.0375
Controlled number of intersections	231	3924	1.6238	1.0198	1.0198	1.0297
Controlled number of intersections	250	4356	1.6606	1.0183	1.0183	1.0275
Controlled number of intersections	283	5140	1.6090	1.0226	1.0226	1.0301
Controlled number of intersections	299	5541	1.6794	1.0076	1.0153	1.0153
Controlled number of intersections	344	7451	1.5714	1.0119	1.0119	1.0357
Controlled number of intersections	395	9639	1.5473	1.0100	1.0149	1.0149
Controlled number of holes	2	22	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	2	16	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	5	22	2.0000	1.0000	1.0000	1.0000
Controlled number of holes	6	34	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	12	64	1.2500	1.0000	1.0000	1.0000
Controlled number of holes	15	111	1.7500	1.0000	1.0000	1.0000
Controlled number of holes	25	152	1.7143	1.0000	1.0000	1.1429
Controlled number of holes	42	235	1.7273	1.0909	1.0909	1.0909
Controlled number of holes	45	336	1.6000	1.0000	1.0000	1.0000
Controlled number of holes	63	361	1.8824	1.1176	1.1176	1.1176
Controlled number of holes	62	475	1.8889	1.1111	1.1111	1.1667
Controlled number of holes	74	621	1.6923	1.0385	1.0385	1.0769
Controlled number of holes	71	511	1.9000	1.1000	1.1000	1.1500
Controlled number of holes	79	772	1.8400	1.0400	1.1200	1.1200
Controlled number of holes	83	597	1.8333	1.1250	1.1250	1.0417
Controlled number of holes	94	868	1.7667	1.0333	1.0667	1.0667
Controlled number of holes	98	1005	1.7429	1.0571	1.0857	1.0857
Controlled number of holes	94	866	1.8065	1.0323	1.0323	1.0323
Controlled number of holes	108	923	1.8000	1.0286	1.0571	1.0857
Controlled number of holes	123	1184	1.7442	1.0233	1.0233	1.0233
Controlled number of holes	126	1481	1.7907	1.0698	1.0930	1.1163
Controlled number of holes	124	1254	1.7381	1.0476	1.0714	1.0714
Controlled number of holes	133	1521	1.7000	1.0200	1.0200	1.0400
Controlled number of holes	142	1484	1.8000	1.0200	1.0200	1.0400
Controlled number of holes	151	1767	1.8491	1.0377	1.0377	1.0189
Controlled number of holes	158	1857	1.8393	1.0000	1.0179	1.0179
Controlled number of holes	163	1974	1.7966	1.0339	1.0508	1.0508
Controlled number of holes	173	2308	1.7273	1.0606	1.0606	1.0758
Controlled number of holes	181	2299	1.7681	1.0290	1.0435	1.0145
Controlled number of holes	204	2302	1.7917	1.0833	1.0833	1.0833
Polyominoes composed solely of squares	2	17	1.0000	1.0000	1.0000	1.0000

Polyominoes composed solely of squares	4	26	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	6	59	1.3333	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	7	63	1.3333	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	10	100	1.6667	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	15	124	1.4000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	22	237	1.3333	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	34	309	1.6364	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	26	315	1.4167	1.0000	1.0000	1.0833
Polyominoes composed solely of squares	24	329	1.3636	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	32	354	1.5385	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	41	452	1.7692	1.0769	1.0769	1.0769
Polyominoes composed solely of squares	51	539	1.8889	1.0000	1.0000	1.1111
Polyominoes composed solely of squares	65	649	1.8571	1.0476	1.0476	1.0952
Polyominoes composed solely of squares	67	652	1.6667	1.0417	1.0417	1.1250
Polyominoes composed solely of squares	71	796	1.7917	1.0833	1.0833	1.0833
Polyominoes composed solely of squares	74	827	1.6923	1.0385	1.0385	1.0385
Polyominoes composed solely of squares	84	880	1.7407	1.0370	1.0370	1.0741
Polyominoes composed solely of squares	108	1007	1.7222	1.0833	1.0833	1.0833
Polyominoes composed solely of squares	92	1008	1.8065	1.0323	1.0323	1.0968
Polyominoes composed solely of squares	93	1107	1.7941	1.0000	1.0294	1.0882
Polyominoes composed solely of squares	70	1140	1.5667	1.0000	1.0333	1.0333
Polyominoes composed solely of squares	83	1143	1.7333	1.0667	1.1000	1.1000
Polyominoes composed solely of squares	132	1222	1.8571	1.0476	1.0714	1.0476
Polyominoes composed solely of squares	130	1324	1.7955	1.0455	1.0455	1.0455
Polyominoes composed solely of squares	112	1450	1.7143	1.0000	1.0000	1.0476
Polyominoes composed solely of squares	124	1553	1.7111	1.0222	1.0444	1.0222
Polyominoes composed solely of squares	133	1682	1.7170	1.0377	1.0566	1.0566
Polyominoes composed solely of squares	141	1700	1.8367	1.0204	1.0408	1.0612
Polyominoes composed solely of squares	145	1750	1.7347	1.0408	1.0408	1.0612
Polyominoes composed solely of T-junctions	1	17	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	2	18	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	12	84	1.2000	1.2000	1.2000	1.2000
Polyominoes composed solely of T-junctions	11	87	1.2000	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	17	124	1.6667	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	24	155	1.5000	1.0000	1.0000	1.1250
Polyominoes composed solely of T-junctions	28	181	1.7778	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	43	300	1.6875	1.0625	1.0625	1.0625
Polyominoes composed solely of T-junctions	52	302	1.6667	1.0556	1.0556	1.0556
Polyominoes composed solely of T-junctions	51	341	1.4762	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	57	426	1.5714	1.0000	1.0476	1.0952
Polyominoes composed solely of T-junctions	66	520	1.5652	1.0000	1.0435	1.0435
Polyominoes composed solely of T-junctions	99	617	1.6364	1.0303	1.0303	1.0606
Polyominoes composed solely of T-junctions	86	638	1.5161	1.0323	1.0323	1.0645
Polyominoes composed solely of T-junctions	103	694	1.6667	1.0556	1.0278	1.0278
Polyominoes composed solely of T-junctions	100	776	1.5135	1.0541	1.0811	1.1081
Polyominoes composed solely of T-junctions	119	802	1.6905	1.0000	1.0238	1.0714
Polyominoes composed solely of T-junctions	145	974	1.7609	1.0435	1.0652	1.0435
Polyominoes composed solely of T-junctions	144	1049	1.5849	1.0377	1.0566	1.0189
Polyominoes composed solely of T-junctions	149	1067	1.5357	1.0357	1.0536	1.0714
Polyominoes composed solely of T-junctions	148	1125	1.6226	1.0566	1.0755	1.0943
Polyominoes composed solely of T-junctions	164	1128	1.7037	1.0185	1.0370	1.0741
Polyominoes composed solely of T-junctions	178	1248	1.6724	1.0517	1.0517	1.0690
Polyominoes composed solely of T-junctions	185	1252	1.6462	1.0154	1.0308	1.0308
Polyominoes composed solely of T-junctions	185	1488	1.5395	1.0263	1.0526	1.0263
Polyominoes composed solely of T-junctions	193	1520	1.6528	1.0417	1.0694	1.0556
Polyominoes composed solely of T-junctions	203	1558	1.6447	1.0395	1.0395	1.0395
Polyominoes composed solely of T-junctions	198	1612	1.5696	1.0506	1.0759	1.0506
Polyominoes composed solely of T-junctions	219	1683	1.6076	1.0380	1.0380	1.0759
Polyominoes composed solely of T-junctions	246	1840	1.6854	1.0562	1.0449	1.0562
Regular randomly generated polyominoes	1	18	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	2	22	1.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	16	83	2.0000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	18	122	1.8000	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	20	150	1.8333	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	28	161	1.7143	1.0000	1.1429	1.1429
Regular randomly generated polyominoes	36	233	2.1111	1.0000	1.0000	1.1111
Regular randomly generated polyominoes	65	339	1.8750	1.0625	1.0625	1.0625
Regular randomly generated polyominoes	62	397	1.7647	1.0588	1.0588	1.0588
Regular randomly generated polyominoes	71	469	1.7895	1.0526	1.1053	1.1579
Regular randomly generated polyominoes	84	535	1.9048	1.0476	1.0952	1.1429
Regular randomly generated polyominoes	82	560	1.8261	1.0435	1.0435	1.0435
Regular randomly generated polyominoes	103	613	1.9615	1.0769	1.0769	1.0769
Regular randomly generated polyominoes	108	785	1.8333	1.0667	1.1333	1.1333
Regular randomly generated polyominoes	129	874	1.9697	1.0303	1.0909	1.1212
Regular randomly generated polyominoes	157	945	1.8750	1.0750	1.1250	1.1750
Regular randomly generated polyominoes	176	1116	2.0000	1.1500	1.1500	1.1500
Regular randomly generated polyominoes	184	1159	1.9787	1.0426	1.0638	1.0851
Regular randomly generated polyominoes	165	1185	1.8511	1.0426	1.0851	1.0426
Regular randomly generated polyominoes	225	1375	1.8644	1.0847	1.1186	1.1017
Regular randomly generated polyominoes	202	1426	1.9245	1.0566	1.0943	1.0755
Regular randomly generated polyominoes	231	1489	1.9508	1.0820	1.0820	1.0656
Regular randomly generated polyominoes	231	1620	1.9344	1.0820	1.0984	1.0656
Regular randomly generated polyominoes	241	1649	2.0167	1.0833	1.1167	1.1333
Regular randomly generated polyominoes	283	1762	1.9286	1.0571	1.0857	1.0857
Regular randomly generated polyominoes	301	2091	2.0135	1.1081	1.1081	1.1216
Regular randomly generated polyominoes	322	2155	2.0500	1.1000	1.1250	1.1125
Regular randomly generated polyominoes	277	2188	1.9610	1.0909	1.1039	1.0909
Regular randomly generated polyominoes	344	2317	2.0465	1.1047	1.1163	1.0698
Regular randomly generated polyominoes	374	2504	2.0111	1.1222	1.1444	1.1333
Tree	3	21	1.0000	1.0000	1.0000	1.0000
Tree	5	23	1.5000	1.0000	1.0000	1.0000
Tree	4	43	1.0000	1.0000	1.0000	1.0000
Tree	5	44	1.0000	1.0000	1.0000	1.0000
Tree	6	74	1.3333	1.0000	1.0000	1.0000
Tree	7	76	1.3333	1.0000	1.0000	1.0000
Tree	7	82	1.3333	1.0000	1.0000	1.0000

Tree	11	113	1.6000	1.0000	1.0000	1.0000
Tree	7	124	1.6667	1.0000	1.0000	1.0000
Tree	12	146	2.0000	1.0000	1.2500	1.0000
Tree	11	165	1.4000	1.0000	1.0000	1.0000
Tree	13	189	1.5000	1.0000	1.0000	1.0000
Tree	15	262	1.1429	1.0000	1.1429	1.0000
Tree	19	359	1.6250	1.0000	1.0000	1.0000
Tree	20	361	1.5556	1.0000	1.0000	1.1111
Tree	19	362	1.3333	1.0000	1.0000	1.0000
Tree	22	381	1.7500	1.0000	1.0000	1.0000
Tree	23	448	1.4545	1.0000	1.0000	1.1818
Tree	26	456	1.4167	1.0000	1.0000	1.0000
Tree	24	521	1.2857	1.0000	1.0000	1.0000
Tree	25	524	1.3846	1.0000	1.0000	1.0000
Tree	31	559	1.3333	1.0000	1.0000	1.0000
Tree	26	560	1.3077	1.0000	1.0000	1.1538
Tree	27	569	1.5833	1.0000	1.0000	1.0000
Tree	31	601	1.6000	1.0000	1.0000	1.0000
Tree	27	722	1.4615	1.0000	1.0769	1.0769
Tree	53	3011	1.3667	1.0000	1.0000	1.0000
Tree	59	4241	1.3824	1.0000	1.0000	1.0000
Tree	64	4998	1.2326	1.0000	1.0000	1.0000
Tree	75	6849	1.2553	1.0000	1.0000	1.0000

B.2 Detailed results for the cases involving $k \in \{5, 7, 10, 13, 16\}$

City Type	city size (# of cubic in poly-omino)	# of gates	K	Algo1/01LP	Algo2/01LP	Algo3/01LP	Algo4/01LP
Controlled number of intersections	5	29	5	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	5	29	7	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	5	29	10	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	5	29	13	1.0000	1.0000	1.0000	1.0000
Controlled number of intersections	5	29	16	1.5000	1.0000	1.0000	1.0000
Controlled number of intersections	20	107	5	1.6667	1.0000	1.0000	1.1667
Controlled number of intersections	20	107	7	1.6000	1.0000	1.0000	1.0000
Controlled number of intersections	20	107	10	1.5000	1.0000	1.0000	1.0000
Controlled number of intersections	20	107	13	1.5000	1.0000	1.0000	1.0000
Controlled number of intersections	20	107	16	1.7500	1.0000	1.0000	1.0000
Controlled number of intersections	46	428	5	1.8125	1.1250	1.1250	1.0625
Controlled number of intersections	46	428	7	1.9231	1.0000	1.0000	1.0769
Controlled number of intersections	46	428	10	2.0000	1.0909	1.0909	1.1818
Controlled number of intersections	46	428	13	1.8000	1.1000	1.1000	1.1000
Controlled number of intersections	46	428	16	1.8000	1.2000	1.2000	1.3000
Controlled number of intersections	79	1084	5	1.5938	1.0938	1.0938	1.0625
Controlled number of intersections	79	1084	7	1.8462	1.0000	1.0000	1.0000
Controlled number of intersections	79	1084	10	2.0000	1.1000	1.1000	1.1000
Controlled number of intersections	79	1084	13	2.1176	1.0588	1.0588	1.1176
Controlled number of intersections	79	1084	16	1.7647	1.0000	1.0000	1.0000
Controlled number of intersections	142	2042	5	1.7736	1.0189	1.0377	1.0377
Controlled number of intersections	142	2042	7	1.8085	1.0213	1.0638	1.0638
Controlled number of intersections	142	2042	10	2.1471	1.0588	1.0588	1.0588
Controlled number of intersections	142	2042	13	2.1379	1.0690	1.0690	1.0690
Controlled number of intersections	142	2042	16	2.2963	1.0370	1.0370	1.0370
Controlled number of intersections	231	3924	5	1.6634	1.0099	1.0198	1.0297
Controlled number of intersections	231	3924	7	1.8462	1.0513	1.0641	1.0641
Controlled number of intersections	231	3924	10	2.0820	1.0820	1.0656	1.0984
Controlled number of intersections	231	3924	13	2.0556	1.0926	1.0926	1.1667
Controlled number of intersections	231	3924	16	2.2273	1.1136	1.1136	1.2045
Controlled number of holes	5	22	5	1.0000	1.0000	1.0000	1.0000
Controlled number of holes	5	22	7	3.0000	1.0000	1.0000	1.0000
Controlled number of holes	5	22	10	2.0000	1.0000	1.0000	1.0000
Controlled number of holes	5	22	13	2.0000	1.0000	1.0000	1.0000
Controlled number of holes	5	22	16	2.0000	1.0000	1.0000	1.0000
Controlled number of holes	42	235	5	1.9091	1.0909	1.0909	1.0909
Controlled number of holes	42	235	7	2.0000	1.1250	1.1250	1.1250
Controlled number of holes	42	235	10	2.0000	1.1429	1.1429	1.1429
Controlled number of holes	42	235	13	2.0000	1.1667	1.1667	1.1667
Controlled number of holes	42	235	16	2.0000	1.0000	1.0000	1.1667
Controlled number of holes	71	511	5	2.0000	1.1000	1.1000	1.1500
Controlled number of holes	71	511	7	1.8125	1.1250	1.1250	1.1250
Controlled number of holes	71	511	10	2.0769	1.1538	1.1538	1.2308
Controlled number of holes	71	511	13	2.1818	1.1818	1.0909	1.0909
Controlled number of holes	71	511	16	2.3000	1.1000	1.1000	1.2000
Controlled number of holes	98	1005	5	1.7429	1.0857	1.0571	1.0857
Controlled number of holes	98	1005	7	1.9259	1.0370	1.0370	1.0370
Controlled number of holes	98	1005	10	2.1905	1.0952	1.0476	1.0952
Controlled number of holes	98	1005	13	2.0000	1.1111	1.1111	1.1111
Controlled number of holes	98	1005	16	2.3750	1.1875	1.1250	1.1250
Controlled number of holes	126	1481	5	1.9070	1.0930	1.0930	1.1163
Controlled number of holes	126	1481	7	2.0000	1.0278	1.0833	1.0556
Controlled number of holes	126	1481	10	2.2500	1.0714	1.0357	1.0714
Controlled number of holes	126	1481	13	2.2083	1.1250	1.1250	1.1667
Controlled number of holes	126	1481	16	2.3000	1.1000	1.1000	1.2000
Controlled number of holes	151	1767	5	1.8868	1.0566	1.0377	1.0189
Controlled number of holes	151	1767	7	2.0238	1.0714	1.0476	1.0952
Controlled number of holes	151	1767	10	2.3000	1.0667	1.0667	1.1333
Controlled number of holes	151	1767	13	2.2593	1.1111	1.1111	1.1481
Controlled number of holes	151	1767	16	2.2083	1.0417	1.0417	1.0417
Polyominoes composed solely of squares	6	59	5	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	6	59	7	1.5000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	6	59	10	2.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	6	59	13	1.0000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	6	59	16	1.5000	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	34	309	5	1.7273	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	34	309	7	2.1250	1.1250	1.1250	1.1250
Polyominoes composed solely of squares	34	309	10	2.1429	1.0000	1.0000	1.1429
Polyominoes composed solely of squares	34	309	13	2.1667	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	34	309	16	1.8333	1.1667	1.0000	1.0000
Polyominoes composed solely of squares	51	539	5	1.7778	1.0000	1.0556	1.1111
Polyominoes composed solely of squares	51	539	7	2.0714	1.0714	1.0000	1.1429
Polyominoes composed solely of squares	51	539	10	1.9167	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	51	539	13	2.1111	1.2222	1.1111	1.2222
Polyominoes composed solely of squares	51	539	16	2.2222	1.0000	1.0000	1.1111
Polyominoes composed solely of squares	74	827	5	1.7308	1.0000	1.0385	1.0385
Polyominoes composed solely of squares	74	827	7	1.9000	1.0500	1.0500	1.0500
Polyominoes composed solely of squares	74	827	10	2.3333	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	74	827	13	1.9286	1.0000	1.0000	1.0000
Polyominoes composed solely of squares	74	827	16	2.0769	1.0000	1.0000	1.0769
Polyominoes composed solely of squares	93	1107	5	1.8235	1.0882	1.0882	1.0882
Polyominoes composed solely of squares	93	1107	7	2.0741	1.0370	1.0370	1.0370
Polyominoes composed solely of squares	93	1107	10	2.1500	1.0500	1.0500	1.0500
Polyominoes composed solely of squares	93	1107	13	2.2941	1.1176	1.1176	1.1765

Polyominoes composed solely of squares	93	1107	16	2.2667	1.1333	1.1333	1.1333
Polyominoes composed solely of squares	130	1324	5	1.8409	1.0455	1.0682	1.0455
Polyominoes composed solely of squares	130	1324	7	2.0000	1.0938	1.0938	1.1250
Polyominoes composed solely of squares	130	1324	10	2.2500	1.1250	1.1250	1.2083
Polyominoes composed solely of squares	130	1324	13	2.3500	1.0500	1.0500	1.0500
Polyominoes composed solely of squares	130	1324	16	2.5294	1.1765	1.1765	1.1765
Polyominoes composed solely of T-junctions	12	84	5	1.2000	1.2000	1.2000	1.2000
Polyominoes composed solely of T-junctions	12	84	7	1.2000	1.0000	1.0000	1.2000
Polyominoes composed solely of T-junctions	12	84	10	1.2500	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	12	84	13	1.2500	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	12	84	16	1.2500	1.0000	1.0000	1.0000
Polyominoes composed solely of T-junctions	43	300	5	1.5625	1.0000	1.0625	1.0625
Polyominoes composed solely of T-junctions	43	300	7	1.7143	1.0714	1.0714	1.0000
Polyominoes composed solely of T-junctions	43	300	10	1.6154	1.0769	1.0769	1.0769
Polyominoes composed solely of T-junctions	43	300	13	1.7273	1.0909	1.0909	1.0909
Polyominoes composed solely of T-junctions	43	300	16	1.6364	1.0909	1.0909	1.1818
Polyominoes composed solely of T-junctions	99	617	5	1.6667	1.0303	1.0303	1.0606
Polyominoes composed solely of T-junctions	99	617	7	1.8519	1.1111	1.1111	1.1481
Polyominoes composed solely of T-junctions	99	617	10	1.8333	1.0417	1.0417	1.0417
Polyominoes composed solely of T-junctions	99	617	13	1.7391	1.0000	1.0000	1.0870
Polyominoes composed solely of T-junctions	99	617	16	1.8571	1.0476	1.0476	1.0952
Polyominoes composed solely of T-junctions	119	802	5	1.7381	1.0238	1.0238	1.0714
Polyominoes composed solely of T-junctions	119	802	7	1.9118	1.0588	1.1176	1.1176
Polyominoes composed solely of T-junctions	119	802	10	1.9310	1.0345	1.0345	1.0690
Polyominoes composed solely of T-junctions	119	802	13	1.8148	1.0741	1.0741	1.1481
Polyominoes composed solely of T-junctions	119	802	16	1.8519	1.1111	1.0741	1.1481
Polyominoes composed solely of T-junctions	148	1125	5	1.6981	1.0755	1.0566	1.0943
Polyominoes composed solely of T-junctions	148	1125	7	1.7674	1.0465	1.0465	1.0930
Polyominoes composed solely of T-junctions	148	1125	10	1.8857	1.1143	1.1143	1.2286
Polyominoes composed solely of T-junctions	148	1125	13	1.9688	1.1250	1.1250	1.1563
Polyominoes composed solely of T-junctions	148	1125	16	2.0313	1.1250	1.1250	1.1250
Polyominoes composed solely of T-junctions	185	1488	5	1.6053	1.0263	1.0395	1.0263
Polyominoes composed solely of T-junctions	185	1488	7	1.7258	1.0484	1.0484	1.0484
Polyominoes composed solely of T-junctions	185	1488	10	1.8824	1.0980	1.0980	1.0980
Polyominoes composed solely of T-junctions	185	1488	13	1.8163	1.0612	1.0612	1.1020
Polyominoes composed solely of T-junctions	185	1488	16	1.8667	1.1111	1.1333	1.1778
Regular randomly generated polyominoes	16	83	5	1.7500	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	16	83	7	1.7500	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	16	83	10	2.0000	1.0000	1.0000	1.3333
Regular randomly generated polyominoes	16	83	13	2.0000	1.0000	1.0000	1.3333
Regular randomly generated polyominoes	16	83	16	1.6667	1.0000	1.0000	1.3333
Regular randomly generated polyominoes	65	339	5	2.0000	1.0625	1.0625	1.1250
Regular randomly generated polyominoes	65	339	7	2.0833	1.0833	1.0833	1.1667
Regular randomly generated polyominoes	65	339	10	2.2222	1.2222	1.1111	1.2222
Regular randomly generated polyominoes	65	339	13	2.1250	1.1250	1.1250	1.1250
Regular randomly generated polyominoes	65	339	16	2.1250	1.0000	1.0000	1.0000
Regular randomly generated polyominoes	103	613	5	2.0385	1.0769	1.0769	1.0769
Regular randomly generated polyominoes	103	613	7	2.2000	1.1500	1.1500	1.1500
Regular randomly generated polyominoes	103	613	10	2.0625	1.0000	1.0000	1.0625
Regular randomly generated polyominoes	103	613	13	2.1538	1.0769	1.0769	1.0769
Regular randomly generated polyominoes	103	613	16	2.2500	1.0833	1.0833	1.0833
Regular randomly generated polyominoes	176	1116	5	2.0500	1.1500	1.1500	1.1500
Regular randomly generated polyominoes	176	1116	7	2.0625	1.0625	1.0625	1.0938
Regular randomly generated polyominoes	176	1116	10	2.0833	1.1250	1.1250	1.1667
Regular randomly generated polyominoes	176	1116	13	2.4737	1.1053	1.1053	1.1579
Regular randomly generated polyominoes	176	1116	16	2.2941	1.1765	1.1176	1.2941
Regular randomly generated polyominoes	202	1426	5	1.9434	1.1132	1.0943	1.0755
Regular randomly generated polyominoes	202	1426	7	2.0750	1.1250	1.1250	1.1750
Regular randomly generated polyominoes	202	1426	10	2.4138	1.1724	1.1724	1.1724
Regular randomly generated polyominoes	202	1426	13	2.2400	1.1200	1.1200	1.1600
Regular randomly generated polyominoes	202	1426	16	2.3182	1.0909	1.0909	1.0909
Regular randomly generated polyominoes	283	1762	5	1.9143	1.0857	1.0714	1.1000
Regular randomly generated polyominoes	283	1762	7	2.1765	1.1569	1.1765	1.1373
Regular randomly generated polyominoes	283	1762	10	2.2051	1.0769	1.1026	1.1026
Regular randomly generated polyominoes	283	1762	13	2.3750	1.1250	1.1250	1.1875
Regular randomly generated polyominoes	283	1762	16	2.3571	1.1071	1.1071	1.1071
Tree	4	43	5	1.0000	1.0000	1.0000	1.0000
Tree	4	43	7	1.0000	1.0000	1.0000	1.0000
Tree	4	43	10	2.0000	1.0000	1.0000	1.0000
Tree	4	43	13	2.0000	1.0000	1.0000	1.0000
Tree	4	43	16	2.0000	1.0000	1.0000	1.0000
Tree	11	113	5	1.4000	1.0000	1.0000	1.0000
Tree	11	113	7	1.4000	1.0000	1.0000	1.0000
Tree	11	113	10	1.4000	1.0000	1.0000	1.0000
Tree	11	113	13	1.5000	1.0000	1.0000	1.0000
Tree	11	113	16	1.5000	1.0000	1.0000	1.0000
Tree	15	262	5	1.4286	1.1429	1.0000	1.0000
Tree	15	262	7	1.3333	1.0000	1.0000	1.0000
Tree	15	262	10	1.5000	1.0000	1.0000	1.0000
Tree	15	262	13	1.6000	1.0000	1.0000	1.0000
Tree	15	262	16	1.8000	1.0000	1.0000	1.0000
Tree	22	381	5	1.8750	1.0000	1.1250	1.0000
Tree	22	381	7	1.8750	1.0000	1.0000	1.0000
Tree	22	381	10	1.7143	1.0000	1.0000	1.0000
Tree	22	381	13	1.8333	1.1667	1.1667	1.1667
Tree	22	381	16	1.5000	1.0000	1.0000	1.0000
Tree	25	524	5	1.4615	1.0000	1.0000	1.0000
Tree	25	524	7	1.9000	1.0000	1.0000	1.0000
Tree	25	524	10	1.8750	1.0000	1.0000	1.0000
Tree	25	524	13	2.5000	1.1667	1.1667	1.3333
Tree	25	524	16	2.3333	1.3333	1.3333	1.3333
Tree	31	601	5	1.4000	1.0000	1.0000	1.0000
Tree	31	601	7	1.6429	1.0000	1.0000	1.0000
Tree	31	601	10	1.5833	1.0000	1.0000	1.0000
Tree	31	601	13	1.8000	1.0000	1.0000	1.0000
Tree	31	601	16	1.8000	1.0000	1.0000	1.0000

Bibliography

[1] Hannah Alpert and Érika Roldán. Art gallery problem with rook and queen vision. *Graphs Comb.*, 37(2):621–642, 2021.

[2] Esther M. Arkin, Alon Efrat, Omrit Filtser, Stephen Kobourov, Jan Kratochvíl, Joseph S. B. Mitchell, and Ariel Rosenberg. The gate-cover problem. In Emilio Di Giacomo and Debajyoti Mondal, editors, *WALCOM: Algorithms and Computation*, pages 483–498. Springer Nature Singapore, 2026.

[3] Rom Aschner, Matthew J. Katz, Gila Morgenstern, and Yelena Yuditsky. Approximation schemes for covering and packing. In *WALCOM: Algorithms and Computation*, 2013.

[4] Stav Ashur, Omrit Filtser, and Matthew J. Katz. A constant-factor approximation algorithm for vertex guarding a wv-polygon, 2019.

[5] Therese Biedl, Timothy M. Chan, Stephanie Lee, Saeed Mehrabi, Fabrizio Montecchiani, and Hamideh Vosoughpour. On guarding orthogonal polygons with sliding cameras. In *WALCOM: Algorithms and Computation*, volume 10167, pages 54–65. Springer, 2017.

[6] Therese Biedl, Mohammad T. Irfan, Justin Iwerks, Joondong Kim, and Joseph S.B. Mitchell. Guarding polyominoes. In *Proceedings of the 27th Annual Symposium on Computational Geometry (SoCG)*, page 387–396. Association for Computing Machinery, 2011.

[7] Therese Biedl and Saeed Mehrabi. On r-Guarding Thin Orthogonal Polygons. In *27th International Symposium on Algorithms and Computation (ISAAC 2016)*, Leibniz International Proceedings in Informatics (LIPIcs), pages 17:1–17:13, 2016.

[8] Sarah Cannon, Thomas G. Fai, Justin Iwerks, Undine Leopold, and Christiane Schmidt. Combinatorics and complexity of guarding polygons with edge and point 2-transmitters. *Computational Geometry*, 68:89–100, 2018.

[9] Paz Carmi, Man-Kwun Chiu, Matthew J. Katz, Matias Korman, Yoshio Okamoto, André van Renssen, Marcel Roeloffzen, Taichi Shiitada, and Shakhar Smorodinsky. Balanced line separators of unit disk graphs. In Faith Ellen, Antonina Kolokolova, and Jörg-Rüdiger Sack, editors, *Algorithms and Data Structures - 15th International Symposium, WADS*, volume 10389, pages 241–252. Springer, 2017.

[10] Timothy M. Chan and Sariel Har-Peled. Approximation algorithms for maximum independent set of pseudo-disks. *Discret. Comput. Geom.*, 48(2):373–392, 2012.

[11] Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. In *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS*, pages 612–623. IEEE, 2022.

[12] Václav Chvátal. A combinatorial theorem in plane geometry. *Journal of Combinatorial Theory, Series B*, 18(1):39–41, 1975.

[13] F. Corradini, M.R. Di Berardini, and W. Vogler. Fairness of components in system computations. *Theoretical Computer Science*, 356(3):291–324, 2006.

[14] Bruno Courcelle. The monadic second-order logic of graphs i: Recognizable sets of finite graphs. *Information and Computation*, 85(1):12–75, 1990.

[15] Ovidiu Daescu and Hemant Malik. New bounds on guarding problems for orthogonal polygons in the plane using vertex guards with halfplane vision. *Theoretical Computer Science*, 882:63–76, 2021.

[16] Minati De and Abhiruk Lahiri. Geometric dominating set and set cover via local search. *CoRR*, abs/1605.02499, 2016.

[17] Mark de Berg, Hans L. Bodlaender, Sándor Kisfaludi-Bak, Dániel Marx, and Tom C. van der Zanden. A framework for exponential-time-hypothesis-tight algorithms and lower bounds in geometric intersection graphs. *SIAM J. Comput.*, 49(6):1291–1331, 2020.

[18] Hubert de Fraysseix, Patrice Ossona de Mendez, and Janos Pach. Representation of planar graphs by segments. *Intuitive Geometry*, 63:109–117, 1991.

[19] Stephan Eidenbenz. Inapproximability results for guarding polygons without holes. In *Algorithms and Computation*, pages 427–437. Springer Berlin Heidelberg, 1998.

[20] Marco Fabris, Riccardo Ceccato, and Andrea Zanella. Efficient sensors selection for traffic flow monitoring: An overview of model-based techniques leveraging network observability. *Sensors*, 25(5), 2025.

[21] Michael R. Fellows, Jan Kratochvíl, Matthias Middendorf, and Frank Pfeiffer. The complexity of induced minors and related problems. *Algorithmica*, 13(3):266–282, 1995.

[22] Omrit Filtser, Erik Krohn, Bengt J. Nilsson, Christian Rieck, and Christiane Schmidt. Guarding polyominoes under k-hop visibility. *Algorithmica*, 87(4):572–593, 2025.

[23] Steve Fisk. A short proof of chvátal's watchman theorem. *Journal of Combinatorial Theory, Series B*, 24(3):374, 1978.

[24] Subir Kumar Ghosh. Approximation algorithms for art gallery problems in polygons. *Discrete Applied Mathematics*, 158(6):718–722, 2010.

[25] Ervin Györi and Tamás Róbert Mezei. Partitioning orthogonal polygons into $\leq$ 8-vertex pieces, with application to an art gallery theorem. *Comput. Geom.*, 59:13–25, 2016.

[26] Frank Harary and Michael Livingston. Characterization of trees with equal domination and independent domination numbers. *Congr. Numer.*, 55:121–150, 1986.

[27] Jeff Kahn, Maria Klawe, and Daniel Kleitman. Traditional galleries require fewer watchmen. *SIAM Journal on Algebraic Discrete Methods*, 4(2):194–206, 1983.

[28] Goos Kant. Drawing planar graphs using the canonical ordering. *Algorithmica*, 16(1):4–32, 1996.

[29] Matthew J. Katz and Gila Morgenstern. Guarding orthogonal art galleries with sliding cameras. *International Journal of Computational Geometry & Applications*, 18(3):241–250, 2008.

[30] James King. Fast vertex guarding for polygons with and without holes. *Computational Geometry*, 46(3):219–231, 2013.

[31] Jan Kratochvíl and Mirko Krivánek. On the computational complexity of codes in graphs. In Michal Chytil, Ladislav Janiga, and Václav Koubek, editors, *Mathematical Foundations of Computer Science 1988, MFCS*, volume 324, pages 396–404. Springer, 1988.

[32] D Lee and Arthurk Lin. Computational complexity of art gallery problems. *IEEE Transactions on Information Theory*, 32(2):276–282, 1986.

[33] David Lichtenstein. Planar formulae and their uses. *SIAM J. Comput.*, 11(2):329–343, 1982.

[34] Anna Lubiw. Decomposing polygonal regions into convex quadrilaterals. In *Proceedings of the first annual symposium on Computational geometry*, pages 97–106, 1985.

[35] Saeed Mehrabi, Ahmad Biniaz, Prosenjit Bose, Anil Maheshwari, and Michiel Smid. Improved bounds for guarding orthogonal art galleries with sliding cameras. *Computational Geometry*, 64:23–35, 2017.

[36] Nabil H. Mustafa and Saurabh Ray. Improved results on geometric hitting set problems. *Discret. Comput. Geom.*, 44(4):883–895, 2010.

[37] Nabil Hassan Mustafa and Saurabh Ray. Ptas for geometric hitting set problems via local search. In *Proceedings of the Twenty-Fifth Annual Symposium on Computational Geometry*, SCG '09, page 17–22, New York, NY, USA, 2009. Association for Computing Machinery.

[38] Bengt J. Nilsson. Approximate guarding of monotone and rectilinear polygons. In Luís Caires, Giuseppe F. Italiano, Luís Monteiro, Catuscia Palamidessi, and Moti Yung, editors, *Automata, Languages and Programming*, pages 1362–1373, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[39] Joseph O'ROURKE. The complexity of computing minimum convex covers for polygons. In *Proc. 20th Annu. Allerton Conf. on Communication, Control, and Computing, Allerton, IL, 1982*, pages 75–84, 1982.

[40] Joseph O'Rourke. *Art gallery theorems and algorithms*. Oxford University Press, Inc., USA, 1987.

[41] Christian Rieck and Christian Scheffer. The dispersive art gallery problem. *Computational Geometry*, 117:102054, 2024.

[42] Ariel Rosenberg. The gate cover problem in thin polyomino simulator. `https://arikrosen.pythonanywhere.com/`.

[43] Dietmar Schuchardt and Hans-Dietrich Hecker. Two np-hard art-gallery problems for ortho-polygons. *Mathematical Logic Quarterly*, 41(2):261–267, 1995.

[44] Amenah S.M. Thabit, Chaker Abdelaziz Kerrache, and Carlos T. Calafate. A survey on monitoring and management techniques for road traffic congestion in vehicular networks. *ICT Express*, 10(6):1186–1198, 2024.

[45] Jorge Urrutia. Art gallery and illumination problems. In J.-R. Sack and J. Urrutia, editors, *Handbook of Computational Geometry*, pages 973–1027. Elsevier, 2000.

[46] Mihalis Yannakakis and Fanica Gavril. Edge dominating sets in graphs. *SIAM J. Appl. Math.*, 38(3):364–372, 1980.

[47] Igor E. Zverovich and Vadim E. Zverovich. An induced subgraph characterization of domination perfect graphs. *J. Graph Theory*, 20(3):375–395, 1995.

תוכן העניינים

1	מבוא	1
2	חומר רקע ועבודות קודמות	4
3	בעיית כיסוי השערים	12
4	הוכחת קושי לבעיית כיסוי השערים	17
5	אלגוריתמים לפתרון בעיית כיסוי השערים	29
6	היוריסטיקות חמדניות ותוצאות סימולציה שלהן לפתרון בעיית כיסוי השערים	36
7	סיכום ומסקנות	48
	נספחים	50
	ביבליוגרפיה	65

תקציר

נוכח הצורך בפתרונות בתחום ניטור ופיקוח עירוני, עבודה זו מציגה וחוקרת וריאנט חדש של בעיית גלריית האמנות הקלאסית, הנקרא **בעיית כיסוי השערים**.

בעיית כיסוי השערים מרחיבה את המסגרת המסורתית של בעיות כיסוי בכך שהיא מתמקדת בכיסוי שערים ייעודיים במקום בפיקוח על כלל הצורה הגיאומטרית. באופן פורמלי, הבעיה מבקשת למקם מספר מינימלי של מצלמות בפוליאומינו דק כך שקבוצה נתונה של שערים תהיה מכוסה. נבחנים שני מקרים עיקריים: מצלמות בעלות טווח ראייה בלתי מוגבל ומצלמות בעלות טווח ראייה מוגבל. הניתוח התאורטי מראה כי שני המקרים הם בעייה NP - קשה. בפרט, המקרה של מצלמות בלתי מוגבלות נותר NP - קשה גם עבור פוליאומינוים המוגבלים לצמתי T בלבד. עבור המקרה של מצלמות עם טווח ראייה בלתי מוגבל, ניתן להראות רדוקציה לבעיית כיסוי בקשתות בגרפים שבהם הצמתים מונחים על רשת סדורה, והוכחה לאלגוריתם קירוב ביחס 2 לפתרון האופטימלי בזמן ליניארי. עבור המקרה של טווח ראייה מוגבל, מובאת הוכחה שהבעייה קשה-NP לכל פרמטר של טווח הראייה $k \geq 1$, והוכחה לסכמת קירוב פולינומית מלאה (PTAS) המבוססת על חיפוש מקומי. מעבר לניתוח התאורטי, נבחנות היוריסטיקות חמדניות, ותוצאות ניסיוניות מראות כי באמצעות אסטרטגיות מתאימות לשבירת שוויון, הן משיגות באופן עקבי ביצועים קרובים לאופטימום במגוון רחב של של ערים המיוצגות על ידי פוליאומינו צר. עבודה זו מקדמת הן את ההבנה התאורטית והן את ארגז הכלים האלגוריתמי לבעיות כיסוי בגאומטריה חישובית, ותורמת שיטות מעשיות הרלוונטיות ליישומים בעולם האמיתי ברשתות ניטור ופיקוח.



האוניברסיטה הפתוחה
המחלקה למתמטיקה ולמדעי המחשב

בעיית כיסוי שערים בפוליאומינו צר

עבודת תזה זו הוגשה כחלק מהדרישות לקבלת תואר
"מוסמך למדעים" .M.sc במדעי המחשב
באוניברסיטה הפתוחה
המחלקה למתמטיקה ומדעי המחשב

על-ידי
אריאל רוזנברג

בהנחיית ד"ר עומרית פילצר

יוני 2026